

PatchOps: Safety-Gated and Abstention-Aware Repair of GitHub Actions Failures Using LLM Agents

Upamanyu Ghosh

243010119

June 2026

PatchOps: Safety-Gated and Abstention-Aware Repair of GitHub Actions Failures Using LLM Agents

**Thesis submitted to Dr. SPM International Institute of Information Technology,
Naya Raipur in partial fulfillment of the requirements for the degree of**

Master of Technology

in

Computer Science and Engineering

by

Upamanyu Ghosh (243010119)

Under the guidance of

Mr. K.G. Atram

Professor of Practice

Department of Data Science and Artificial Intelligence

Dr. SPM International Institute of Information Technology, Naya Raipur



**Dr. SPM International Institute of Information Technology, Naya
Raipur**

Naya Raipur, India 493661

June 2026

© 2026 Upamanyu Ghosh. All rights reserved.

Dedicated to everyone whose patience, support, and belief made this work possible.

Approval

This thesis titled **PatchOps: Safety-Gated and Abstention-Aware Repair of GitHub Actions Failures Using LLM Agents** has been reviewed in draft form and is approved as a candidate dissertation for the degree of **Master of Technology in Computer Science and Engineering**.

Mr. K.G. Atram

Professor of Practice

Department of Data Science and
Artificial Intelligence

Dr. SPM International Institute
of Information Technology, Naya
Raipur

Examiner / Committee

Member

To be nominated by the institute
Dr. SPM International Institute
of Information Technology, Naya
Raipur

Date: 30th June 2026

Place: Naya Raipur

Declaration

I hereby declare that the work reported in this thesis, entitled **PatchOps: Safety-Gated and Abstention-Aware Repair of GitHub Actions Failures Using LLM Agents**, is an original record of research carried out by me under the supervision of **Mr. K.G. Atram**. To the best of my knowledge, this work has not been submitted, either in part or in full, to any other institution for the award of any degree or diploma.

Upamanyu Ghosh

243010119

Computer Science and
Engineering

Department of Computer
Science and Engineering

Date: 30th June 2026

Place: Naya Raipur

Certificate

This is to certify that the thesis entitled **PatchOps: Safety-Gated and Abstention-Aware Repair of GitHub Actions Failures Using LLM Agents** submitted by **Upamanyu Ghosh (243010119)** to **Dr. SPM International Institute of Information Technology, Naya Raipur** for the award of the degree of **Master of Technology in Computer Science and Engineering** is a bona fide record of research work carried out under my supervision.

Mr. K.G. Atram
Professor of Practice
Department of Data Science
and Artificial Intelligence
Dr. SPM International
Institute of Information
Technology, Naya Raipur

Date: 30th June 2026

Place: Naya Raipur

Plagiarism Report

This page is reserved for the official plagiarism report required by the institute. The final report may be attached here after the similarity check is completed.

Similarity Index: To be inserted after the official institutional similarity check.

Report Date: 30th June 2026.

Acknowledgment

I would like to express my sincere gratitude to **Mr. K.G. Atram** for guidance, patience, and constructive feedback throughout this work. The structure and discipline of this thesis benefited directly from that support.

I am equally thankful to my peers, collaborators, and everyone who contributed feedback on the PatchOps project, the evaluation methodology, and the thesis narrative. Their questions helped sharpen both the system and the claims made in this dissertation.

Finally, I am grateful to my family for their constant encouragement and support, without which this work would not have been possible.

Upamanyu Ghosh

243010119

Computer Science and
Engineering

30th June 2026
Naya Raipur

Abstract

PatchOps is a repository-aware, safety-gated, abstention-aware repair system for GitHub Actions failures. It combines log extraction, failure classification, fault localization, repair planning, patch generation, and a closed CI validation loop to decide whether the system should open a fix pull request, request review, rerun the workflow, escalate, or abstain.

The thesis investigates not only whether an AI system can repair CI failures, but under what conditions it should decline to act. Across the evaluated method families, fault localization remains the dominant bottleneck, with Recall@3 staying below 0.45 and peaking at 0.32. The results also show that improved localization is necessary but not sufficient for successful repair: full-repository retrieval can raise patch-apply rate to 1.00 on the gated path without lifting CI Pass@1 above 0.00 on that same path.

PatchOps is therefore positioned not as a state-of-the-art repair engine on raw Pass@1 alone, but as a reproducible and safety-conscious repair workflow that emphasizes honesty, calibrated decision-making, and closed-loop validation. On the original 20-case Safety Set it records 0/20 unsafe auto-fixes while maintaining high abstention accuracy, and on the expanded 76-case decision-gate study the learned calibrated act-versus-hold policy reaches 0.91 act/hold accuracy versus 0.855 for the earlier hand-set threshold policy. This framing aligns the system with the practical needs of software engineering teams that value dependable automation over aggressive but unsafe patch generation.

Keywords: abstention, CI repair, GitHub Actions, large language models, safety gate.

Contents

Approval	9
Declaration	11
Certificate	13
Plagiarism Report	15
Acknowledgment	i
Abstract	iii
List of Abbreviations	xi
List of Symbols and Notations	xiii
1 Introduction	1
1.1 Research Context and Gap	2
1.2 PatchOps Thesis Position	2
1.3 Problem Motivation	3
1.4 Research Questions	4
1.5 Contributions	4
1.6 Thesis Organization	5
1.7 Chapter Conclusion	5
2 Background and Related Work	7
2.1 Introduction	7
2.2 LLM-Based CI/CD Diagnosis and Remediation	7
2.3 Automated Program Repair and Build Repair	8
2.4 Why CI Repair Is Not the Same as Classical APR	9
2.5 Overfitting, Plausibility, and Reward Hacking	9
2.6 Benchmarks and Datasets	10
2.7 Failure Taxonomy and the Diversity of CI Failures	11
2.8 Flaky and Transient Failures	11
2.9 Industry and Open-Source Landscape	12
2.10 Implications for PatchOps Design	12
2.11 Chapter Conclusion and Positioning	13

3	Problem Statement	15
3.1	Introduction	15
3.2	Task Definition	15
3.3	Decision Space	16
3.4	Failure Taxonomy and Risk Framing	16
3.5	Confidence, Learned Decision Gate, and Intended Actions	17
3.6	Semantic Ambiguity and Human Judgement	18
3.7	Safety Requirements	19
3.8	Problem Boundaries and Scope	20
3.9	Non-Goals	20
3.10	From Problem Statement to Architecture	21
3.11	Chapter Conclusion	21
4	System Design	23
4.1	Introduction	23
4.2	Architectural Philosophy	23
4.3	Current Implemented Workflow	24
4.4	Stage 1: Log Extraction	25
4.5	Stage 2: Failure Classification	25
4.6	Stage 3: Flaky and Transient Detection	25
4.7	Stage 4: Repository Context Retrieval	26
4.8	Stage 5: Root-Cause Analysis	26
4.9	Stage 6: Patch Planning and Intended Action	26
4.10	Stage 6b: Learned Decision Gate	27
4.11	Stage 7: Semantic Ambiguity Backstop	28
4.12	Stage 8: Patch Generation	28
4.13	Stage 9: Safety Gate	29
4.14	Stage 10: Plausibility Gate	29
4.15	Final Decision and Trace	29
4.16	Current Workflow as a Figure	30
4.17	Validation Harness and Target Agentic Architecture	32
4.18	Implemented Extensions and Target Growth	32
4.19	Why the Architecture Matches the Thesis Claim	33
4.20	Chapter Conclusion	33
5	Implementation	35
5.1	Introduction	35
5.2	Implementation Structure	35
5.3	The Engine Entry Point	36
5.4	Evidence Compression and Classification	36

5.5	Provider-Agnostic LLM Layer	36
5.6	Repository Context and Localization	37
5.7	Root-Cause Analysis and Patch Planning	37
5.8	Patch Generation	38
5.9	Safety and Plausibility Controls	38
5.10	Learned Classifier Integration	38
5.11	Validation Harness	39
5.12	Benchmarks as First-Class Implementation Clients	39
5.13	Acceptance and Regression Tests	39
5.14	Implementation Principles	40
5.15	Chapter Conclusion	40
6	Methodology	43
6.1	Introduction	43
6.2	Evaluation Philosophy	43
6.3	Phase-by-Phase Pre-Registration	44
6.4	Datasets and Evaluation Sources	44
6.5	Why a Multi-Source Design Is Necessary	45
6.6	Research Questions and Evaluation Axes	45
6.7	Baselines and Ablation Ladder	46
6.8	Validation Strategy	46
6.9	Metric Families	47
6.10	Formal Metric Definitions	48
6.10.1	Localization Metrics	48
6.10.2	Repair Metrics	48
6.10.3	Classification Metrics	49
6.10.4	Decision and Abstention Metrics	49
6.10.5	Selective Prediction and Calibration	49
6.10.6	Plausibility and Ranking Quality	50
6.10.7	Confidence Intervals	50
6.11	Ground-Truth Design for Safety and Flakiness	50
6.12	Localization Methodology as a Separate Study	51
6.13	Classifier Methodology as a Reproducibility Study	51
6.14	Selective Prediction and Calibration Methodology	51
6.15	Learned Decision-Gate Methodology	51
6.16	Agentic Localization Methodology	52
6.17	Threats to Validity Built Into the Methodology	52
6.18	Methodological Honesty Rules	53
6.19	Chapter Conclusion	53

7	Results	55
7.1	Introduction	55
7.2	Diagnosis Quality and Early Benchmark Behavior	55
7.3	Fault Localization Remains Hard Across Tested Methods	57
7.3.1	Log-Only Localization Strategies	57
7.3.2	Whole-Repository Retrieval	57
7.3.3	Semantic Embedding Retrieval	58
7.3.4	Bounded Agentic Exploration	58
7.3.5	Localization Conclusion	59
7.4	Better Localization Is Necessary but Not Sufficient for Repair	60
7.4.1	Supporting Result: Learned Classification as a Reproducibility and Cost Result	62
7.5	CI-Green Is Not Enough	62
7.5.1	Reward Hacking	63
7.5.2	Patch Plausibility	63
7.6	Abstention Must Be Engineered	64
7.6.1	Selective Prediction and Calibration	64
7.6.2	The Learned Calibrated Decision Gate	66
7.6.3	Safety Set Evaluation	67
7.6.4	Self-Abstention Nuance	67
7.7	Safety and Practicality as System Outcomes	68
7.8	Result Synthesis	69
7.9	Chapter Conclusion	69
8	Discussion	71
8.1	Introduction	71
8.2	Why the Negative Results Matter	71
8.3	PatchOps as a Conditional Repair System	72
8.4	Why Abstention Must Be Engineered	72
8.5	CI-Green as a Partial Oracle	73
8.6	The Role of the Learned Classifier	73
8.7	The Learned Decision Gate as the Abstention Capstone	74
8.8	Architecture: Why a Staged Pipeline Is Preferable Here	74
8.9	Implications for Deployment	75
8.10	Relation to Prior CI Repair Work	76
8.11	Chapter Conclusion	76
9	Limitations	77
9.1	Introduction	77
9.2	Construct Validity	77

9.2.1	CI Pass Is Not Full Correctness	77
9.2.2	File-Overlap Is a Proxy for On-Target Repair	78
9.2.3	Localization Metrics Capture Ranking, Not Understanding . . .	78
9.2.4	Safety Labels Are Operational	78
9.3	Internal Validity	78
9.3.1	Pipeline Bugs and Regression Risk	79
9.3.2	Model Non-Determinism	79
9.3.3	Local CI Reproduction	79
9.3.4	Budget and Provider Constraints	79
9.4	External Validity	80
9.4.1	Python-First Benchmark Scope	80
9.4.2	GitHub Actions Scope	80
9.4.3	Benchmark Representativeness	80
9.4.4	Synthetic-Heavy Decision-Gate Data	80
9.4.5	Repository Size and Complexity	81
9.5	Conclusion Validity	81
9.5.1	Sample Sizes	81
9.5.2	Multiple Experiments and Narrative Selection	81
9.5.3	Threshold Choices	82
9.6	Practical System Limitations	82
9.7	Chapter Conclusion	82
10	Conclusion	83
10.1	Conclusion	83
10.2	Summary of Contributions	83
10.3	Answers to the Research Questions	84
10.4	Future Work	85
10.5	Final Remarks	86
A	Reproducibility Artifacts	87
A.1	Purpose	87
A.2	Canonical Result Sources	87
A.3	Benchmark and Analysis Scripts	87
A.4	Engine Modules	88
	References	91
	Publication	93
	Author's Biography	95

List of Abbreviations

AUC	Area Under the Receiver Operating Characteristic Curve
APR	Automated Program Repair
AURC	Area Under the Risk-Coverage Curve
BM25	Best Matching 25 lexical ranking model
CI	Continuous Integration
CV	Cross-Validation
ECE	Expected Calibration Error
F1	Harmonic mean of precision and recall
LOO	Leave-One-Out evaluation
LLM	Large Language Model
PR	Pull Request
RCA	Root Cause Analysis
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
Recall@k	Fraction of cases localized within the top- k ranked files
Pass@1	Fraction of benchmark instances solved by the first generated repair
ROC	Receiver Operating Characteristic
RQ	Research Question
TF-IDF	Term Frequency-Inverse Document Frequency

List of Symbols and Notations

N	Number of evaluated benchmark instances.
k	Rank cutoff used in Recall@ k and Pass@ k metrics.
x_i	Feature vector for instance i before decision-gate prediction.
$p_{i,\text{cal}}$	Calibrated probability that PatchOps should act on instance i .
τ	Decision threshold used by the calibrated act-versus-hold gate.
ECE	Expected calibration error.
AURC	Area under the risk-coverage curve.

CHAPTER 1

Introduction

Preface

This chapter introduces the PatchOps thesis, motivates the CI-repair problem, defines the core research gap, and states the thesis contribution around safety, abstention, and honest CI-validated automation.

Continuous integration and continuous deployment (CI/CD) pipelines are now part of everyday software development. In modern repositories, every pull request, dependency update, and workflow change can trigger a GitHub Actions run that either confirms the health of the system or exposes a failure that blocks development. Although these failures are valuable signals, they are also expensive in practice. Developers must inspect raw logs, determine whether the failure is caused by configuration drift, dependency mismatch, test regressions, environment problems, or transient infrastructure issues, and then decide whether the right response is to patch code, re-run the workflow, or escalate the issue for human review. This triage work is repetitive, time consuming, and often urgent because failed CI blocks merges, delays releases, and interrupts developer flow.

Recent progress in large language models (LLMs) has made it possible to automate parts of this process [1, 2]. Log-analysis systems can summarize failures and suggest likely causes, while repository-aware repair systems can attempt patches and validate them by re-running CI. However, the practical problem is not simply whether a model can generate a patch. In real engineering workflows, an inappropriate automatic fix can be worse than no fix at all. A tool that edits the wrong file, suppresses a failing test, disables a check, or proposes a risky change for a transient failure may turn a local CI incident into a broader reliability problem. For this reason, the central question of this thesis is not “can an LLM make CI green?” but rather “what is the right action for a given CI failure, and how can that action be taken safely?”

This thesis presents **PatchOps**, a repository-aware AI agent for GitHub Actions failures that combines failure diagnosis, repository context retrieval, patch generation, local CI re-validation, and explicit safety-gated decision making. Instead of treating every failure as an invitation to generate a patch, PatchOps frames CI repair as a decision under uncertainty. Given a failed workflow run and its repository context, the system decides among several actions: open a patch pull request, present a preview for human review, recommend a re-run for likely transient failures, escalate risky cases, or abstain when evidence is insufficient. This decision-centric framing is the thesis backbone.

1.1 Research Context and Gap

The research landscape already contains strong prior work on CI failure analysis and repository-level repair. In particular, CI-Repair-Bench establishes CI-validated repository repair as a benchmarked task [2], while industrial systems such as LogSage demonstrate that LLM-based diagnosis can be useful at scale [1]. As a result, PatchOps does not claim to be the first system to perform CI-verified repair, nor does it claim to surpass the state of the art in raw repair success. Such claims would be inaccurate and would obscure the actual research opportunity.

The genuine gap addressed by this thesis is different. Existing CI repair systems largely optimize for whether a patch can be generated and whether CI turns green after that patch is applied. They do not place equal emphasis on whether the agent should patch in the first place, how often it should refuse to act, whether a green result is obtained honestly, or whether the selected patch is even targeted at the right part of the repository. In other words, the literature has established CI repair as a task, but it has not adequately framed repair as a *risk-sensitive decision process*. This is where PatchOps contributes: it adds a safety and abstention layer on top of an already established repair setting, and evaluates that layer explicitly rather than treating it as an implementation detail.

1.2 PatchOps Thesis Position

PatchOps demonstrably repairs CI failures end to end: it produces CI-verified fixes that turn failed workflows green, shown on real GitHub pull requests whose checks passed after the patch. The repair capability is therefore not in question. The position taken in this thesis builds on that capability rather than disputing it. PatchOps is claimed as a reproducible, safety-gated, abstention-aware CI-repair agent that is dependable when failures are log-localizable and cautious when they are not. The main contribution is consequently not raw patch volume, aggressive automation, or a higher repair rate than prior work; it is the engineering of *trustworthy* repair — knowing *when not to act* — together with a closed validation loop that checks proposed fixes through CI-style re-execution before they are treated as credible.

The current evidence base in the project supports four findings that define the thesis:

1. Fault localization is the binding constraint, and within the methods tested it is not closed by retrieval alone. Across four strategies evaluated in this work — lexical BM25, LLM reranking, semantic embedding retrieval, and bounded agentic (ReAct) exploration — recall improves only modestly and plateaus below the target bar needed for robust repair. We therefore argue, for our setting, that simple retrieval is insufficient and that the residual difficulty appears to be one of repository reasoning; we do not claim this as a universal property of CI repair.

2. Better localization is necessary but not sufficient for successful repair. Full-repository retrieval can remove patch-application bottlenecks without producing corresponding gains in end-to-end Pass@1 repair success.
3. A green CI result is not automatically a correct result. Some patches can be off-target or reward-hacking in nature, so honesty and plausibility checks are required in addition to validation.
4. Abstention cannot be left to the base agent alone. A tool-using agent may rarely self-decline even when it should, so abstention must be engineered through explicit selective controls, safety gates, and the learned calibrated act-versus-hold gate introduced in the final system.

These findings are valuable even when they are negative or partially negative. A central strength of the PatchOps study is that it does not treat honest negative results as failures to hide. Instead, they are used to separate what the system already does well from what remains unsolved in CI repair, especially semantic repair after the correct file has already been identified.

1.3 Problem Motivation

The motivation for PatchOps comes from the mismatch between the operational reality of CI failures and the way automated repair is often discussed. Not all failures should be patched. Some runs fail because of flaky tests, network issues, missing credentials, permission errors, deployment conditions, or ambiguous semantic regressions. In such cases, blindly generating a patch is either unhelpful or unsafe. A production-relevant repair assistant therefore needs at least three properties.

First, it must be **repository-aware**. CI failures are rarely understandable from logs alone. Workflow files, dependency manifests, configuration files, and the surrounding codebase provide crucial context for understanding what actually broke.

Second, it must be **validation-aware**. A generated patch should not be treated as useful merely because it looks plausible in text. Practical repair demands a feedback loop in which the patch is applied and the relevant CI commands are re-executed.

Third, it must be **safety-aware**. The system must recognize risky conditions, avoid touching forbidden or sensitive files, and prefer preview, re-run, or escalation modes over automation when evidence is weak or the stakes are high.

PatchOps is designed around these three requirements. The resulting system is not a general autonomous software engineer; it is a scoped CI-repair agent whose primary value lies in disciplined decision making under noisy and incomplete evidence.

1.4 Research Questions

This thesis is organized around five fixed research questions:

1. Does repository-aware context improve CI/CD failure diagnosis and fault localization compared with log-only prompting?
2. Does PatchOps generate patches that apply cleanly and improve CI repair success compared with simpler LLM baselines?
3. Does a safety-gated decision policy reduce unsafe or inappropriate repair attempts?
4. Can PatchOps distinguish repairable failures from transient or flaky failures where re-running is a better action than patching?
5. What is the cost and latency of running PatchOps in a realistic GitHub workflow?

These questions keep the work grounded. They separate diagnosis from repair, repair from safety, and automation from practicality. They also ensure that the thesis does not collapse into a vague “AI for DevOps” claim.

1.5 Contributions

The contributions of this thesis are summarized as follows:

1. **A system contribution:** PatchOps, an end-to-end repository-aware repair agent for GitHub Actions failures that integrates diagnosis, localization, patch planning, generation, validation, and decision making.
2. **A methodological contribution:** a staged repair architecture that treats CI repair as a risk-sensitive decision process rather than a patch-generation task alone.
3. **A safety contribution:** explicit support for patch, preview, re-run, escalate, and abstain actions, along with safety-gated controls over automated behavior.
4. **An evaluation contribution:** a benchmarked study that reports not only diagnosis and repair metrics, but also unsafe-edit rate, abstention accuracy, calibration-oriented selective metrics, and patch-plausibility indicators.
5. **An empirical contribution:** a set of findings showing that localization improvements alone do not solve CI repair, that green CI outcomes require honesty checks, and that abstention must be engineered rather than assumed.

1.6 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the background and related work on LLM agents, CI failure diagnosis, automated program repair, safety, calibration, and abstention. Chapter 3 defines the problem, scope, and safety requirements of PatchOps. Chapter 4 presents the system design and architecture, including the decision pipeline and validation loop. Chapter 5 describes the implementation of the PatchOps engine and its supporting modules. Chapter 6 explains the experimental methodology, datasets, baselines, and evaluation metrics. Chapter 7 reports the empirical results, structured around the thesis’s four headline findings. Chapter 8 discusses the broader meaning of these findings and their implications for practical CI repair agents. Chapter 9 examines threats to validity and current limitations. Chapter 10 concludes the thesis and outlines future research directions, with particular emphasis on semantic repair and stronger verification.

1.7 Chapter Conclusion

This chapter has introduced the PatchOps thesis, its motivation, research questions, and contributions. It has also established the central framing used throughout the document: CI repair is not only a patch-generation problem, but a risk-sensitive decision problem requiring repository context, validation, safety gates, and engineered abstention. The next chapter situates this framing within the related literature.

CHAPTER 2

Background and Related Work

Preface

This chapter reviews the research context for PatchOps, including CI/CD failure diagnosis, automated program repair, repository-aware repair benchmarks, safety, calibration, and abstention.

2.1 Introduction

The PatchOps problem sits at the intersection of several active research areas rather than belonging to only one of them. It is related to log-based root-cause analysis, automated program repair, repository-level software engineering agents, CI benchmark construction, failure taxonomy studies, and the growing literature on calibration, abstention, and overfitting in LLM-based software systems [1, 5, 15, 13]. A good related-work chapter for PatchOps therefore needs to do more than list nearby papers. It must explain which parts of the prior literature PatchOps inherits directly, which claims have already been established by others, and which gap still remains meaningful enough to support a thesis contribution.

As of mid-2026, the field has moved beyond the stage where one can claim novelty simply by saying that an LLM reads CI logs and proposes a fix. Industrial and academic systems already demonstrate that CI diagnosis and even CI-validated repository repair are feasible tasks. The important question is no longer whether repair should be attempted at all, but rather how such repair should be *framed, controlled, and evaluated*. This chapter reviews the most relevant prior work and shows why PatchOps is best positioned not as “the first CI repair agent,” but as a safety-gated, abstention-aware, closed-loop repair system built on top of an already established task.

2.2 LLM-Based CI/CD Diagnosis and Remediation

The closest diagnosis-oriented system to PatchOps is **LogSage** (ASE 2025), an industrially validated framework for CI/CD failure detection and remediation. LogSage combines an offline knowledge-building phase with an online diagnosis phase in which failed logs are filtered, expanded, and pruned before being passed to LLM prompts supported by retrieval [1]. Its core strength is not repository patch generation, but industrial-scale root-cause analysis with high reported precision and a large deployment footprint. For

PatchOps, LogSage is important for two reasons.

First, it establishes that log preprocessing and focused evidence extraction matter. Raw CI logs are noisy, redundant, and often too large to feed directly to an LLM. PatchOps inherits this lesson through its log extraction stage and should be understood as part of the same design tradition: extract the critical signal before asking a model to reason.

Second, LogSage sets a strong baseline in the diagnosis portion of the problem. It would be unrealistic and unnecessary for PatchOps to compete with a year-long industrial deployment on the narrow question of root-cause analysis precision. The more useful interpretation is that systems such as LogSage solve an important part of the diagnosis problem, while leaving open the question of how diagnosis should connect to repository edits, patch validation, and safety-sensitive action selection.

The most directly relevant repair-oriented work is the reference workflow introduced with **CI-Repair-Bench** (arXiv, April 2026) [2]. Its pipeline follows a simple but consequential structure: analyze the CI log, localize the likely fault, generate a patch, and then re-run the original workflow to measure whether the fix succeeds under true CI-style validation. This benchmark is a turning point for the PatchOps thesis because it already establishes repository-aware, CI-verified repair as a public task. It also provides a concrete performance reference, with the strongest baseline still achieving only around one-fifth Pass@1 overall. In other words, CI repair has been formalized, but it remains far from solved.

This matters for thesis positioning. PatchOps cannot honestly claim to be the first system to validate patches through CI, nor can it claim that repository-aware patch repair itself is unexplored. What remains open is what CI-Repair-Bench does not emphasize: when an agent should decline to patch, how it should behave on transient or risky failures, and how to prevent superficially successful but unsafe or off-target edits from being treated as wins. That reframing is central to PatchOps.

2.3 Automated Program Repair and Build Repair

PatchOps also belongs to the longer history of automated program repair (APR), especially the branch of APR concerned with build and compilation failures. Earlier systems such as **HireBuild** and **DeepDelta** addressed narrower but structurally similar tasks: they used mined fix patterns or learned transformations to repair failing build scripts or compilation errors. These systems demonstrated that repeated software failure patterns can often be corrected automatically, especially when the failure class is narrow and the repair space is constrained [3, 4].

Their relevance to PatchOps is historical and conceptual. They show that repair is not a brand-new ambition, but they also expose the limitations of narrow repair regimes. Build-script repair, compilation repair, and configuration repair work best when the failure type is predictable and the patch space is small. CI repair is harder because CI failures

are broader in scope: they may arise from code, tests, dependencies, workflow YAML, environment setup, permissions, missing secrets, or flaky infrastructure. PatchOps therefore inherits the repair ambition of pre-LLM systems, but it operates in a setting that is more heterogeneous and less structurally constrained.

The broader LLM-APR field makes this contrast even clearer. Surveys of LLM-based APR show fast progress in patch generation, but they also show that success depends heavily on task framing, benchmark contamination, validation quality, and the ability to localize the right files before editing them. Repository-level agentic benchmarks such as SWE-bench and its descendants show that well-structured pipelines can outperform overly complex multi-agent designs [5, 6, 7, 8]. This is an important design lesson for PatchOps. A staged state-machine pipeline, with explicit transitions from log evidence to localization, patch planning, generation, validation, and final decision, is not merely an engineering convenience. It is a research-informed choice that aligns with evidence that simpler, structured repair workflows are often easier to reproduce, analyze, and ablate than multi-agent systems built for appearance rather than clarity.

2.4 Why CI Repair Is Not the Same as Classical APR

Although PatchOps is related to APR, CI repair should not be reduced to a variant of classical test-suite-based program repair. Traditional APR usually assumes that a developer test suite exists, that one can execute it repeatedly, and that patch correctness is approximated by whether tests pass. CI repair is more operational and more fragile. It may involve infrastructure scripts, package installation, linter configuration, workflow permissions, or environmental mismatches rather than source-code semantics alone. A failing GitHub Actions run is therefore not simply a failed unit test. It is a signal produced by a workflow that may stop at the first failure and may expose only a partial view of the system state.

This distinction becomes especially important when considering correctness. Classic APR literature has spent years documenting the gap between a *plausible* patch and a *correct* patch [6, 13, 14]. The same warning applies, and perhaps applies even more strongly, to CI repair. A patch that makes CI green is not automatically a good patch. It may silence a check, change the wrong file, overfit to the immediate failure, or exploit a weakness in the workflow itself. Because CI validation is a partial oracle, green should be interpreted as necessary but not sufficient evidence. This is one of the strongest motivations for PatchOps’ safety gate, patch-plausibility logic, and abstention behavior.

2.5 Overfitting, Plausibility, and Reward Hacking

One of the most stable lessons from APR research is that passing the available tests is not the same as fixing the underlying defect. Multiple studies in both classical APR and

LLM-based APR report high rates of overfitting, where a patch satisfies the visible oracle while remaining semantically wrong or behaviorally fragile. More recent LLM repair studies continue to confirm the same pattern: some patches pass benchmark tests yet fail stronger developer suites, and some even alter behavior in unintended ways while still looking plausible under the benchmark evaluation protocol [6, 13, 14].

For PatchOps, this literature is not background trivia. It directly motivates the decision to treat safety and honesty as first-class research concerns. If the field already knows that plausible patches can be wrong, then a CI repair system that automatically proposes or opens patches without explicit safeguards is not just incomplete; it is methodologically naive. PatchOps responds to this by adding layers that go beyond basic patch generation: reward-hack detection, patch-plausibility scoring, minimality-oriented gating, and abstention paths for ambiguous cases.

This also helps explain why the thesis does not frame safety as a product add-on. Safety is not merely a user experience preference. It is a scientific necessity forced by the known limitations of CI and APR evaluation oracles. In this sense, the PatchOps safety gate is a direct descendant of the overfitting debate in APR.

2.6 Benchmarks and Datasets

The choice of benchmark shapes what claims can be made. For PatchOps, the most important benchmark is **CI-Repair-Bench**, which offers 567 CI failure instances across more than one hundred Python repositories, organized into twelve categories and validated through full workflow re-execution. Its main value is that it turns PatchOps into a comparable system rather than an isolated prototype [2]. A thesis evaluated on CI-Repair-Bench can be discussed alongside existing repair baselines, while a thesis evaluated only on a private demo set would be much harder to defend.

However, no single dataset is enough for all PatchOps questions. CI-Repair-Bench is strong for end-to-end repair success but weaker for the kinds of safety claims that matter most to this thesis. It does not richly label risky conditions such as missing secrets, deployment permissions, or transient failures where rerunning is better than patching. This is why PatchOps also relies conceptually on controlled safety and fault-injection cases, as well as on broader CI log corpora such as **GHALogs**, which are useful for classification and distributional context even if they do not directly support patch validation.

The literature also highlights the value of reproducible CI execution tooling, such as **GitBug-Actions**, which shows how GitHub Actions workflows can be replayed locally in controlled environments. This line of work matters because practical CI evaluation is often bottlenecked not by the patch generator, but by the difficulty of re-executing workflows repeatedly, cheaply, and reproducibly. PatchOps fits naturally into this methodological space: its validation loop is meaningful only if the execution

environment can be treated as dependable enough for repeated evaluation [9, 10].

2.7 Failure Taxonomy and the Diversity of CI Failures

Another important strand of related work studies why GitHub Actions workflows fail in the first place. Empirical taxonomy work, including the large-scale TOSEM 2025 study of GitHub Actions failures, shows that CI failures are diverse and cannot be reduced to one or two failure modes. Project-level configuration errors, dependency problems, runtime issues, environmental setup failures, permissions, and third-party service interactions all appear in practice [11]. This diversity is essential for PatchOps because it justifies both the failure classifier and the decision policy.

If all failures were simply code-edit problems, then an always-patch system might be a defensible first approximation. The literature shows the opposite. CI failures include many situations where the safest or most appropriate action is not a patch. This is why PatchOps organizes failures into operational categories and why the thesis places importance on taxonomy crosswalks between the project’s own labels, CI-Repair-Bench’s categories, and broader empirical taxonomies from the literature.

This taxonomy perspective also strengthens the thesis gap. Public benchmarks emphasize repairable failures, but the operational world includes many cases that are better treated as rerun, preview, or escalate. PatchOps turns this mismatch into a contribution by explicitly modeling those decision classes rather than collapsing all failures into the same repair workflow.

2.8 Flaky and Transient Failures

Research on flaky tests and transient CI failures provides one of the strongest motivations for PatchOps’ abstention-aware behavior. Studies on GitHub Actions flakiness show that some failures arise from instability in external services, timing effects, environment differences, network issues, or order-dependent test behavior. In such cases, producing a source-code patch is often the wrong response. A rerun, quarantine, or human review is safer and more informative [12].

This is a crucial distinction between PatchOps and simple repair pipelines. Most repair benchmarks reward only successful patches; they do not reward the model for correctly deciding *not* to patch. PatchOps instead treats flaky detection and abstention as measurable successes. This creates a different notion of competence: the system is not judged solely by how often it edits, but by how often it chooses the right operational action.

From a thesis perspective, this is one of the clearest novelty lanes available. Correctly distinguishing a repairable failure from a transient one is both practically valuable and under-measured in prior CI repair evaluations. It also produces compelling demos and

easy-to-understand motivation, because a junk PR for a network timeout is obviously less desirable than a correct rerun recommendation.

2.9 Industry and Open-Source Landscape

The broader industry landscape confirms that the product idea of “AI fixes CI” is no longer novel by itself. GitHub Copilot Autofix, Dependabot, Renovate, Copilot coding-agent workflows, and multiple engineering blog posts on self-healing CI all point in the same direction: software teams increasingly expect automated help when pipelines fail. Some systems focus on vulnerabilities or dependency updates, some are issue-driven coding agents, and others wire LLMs into branch-level repair loops [1].

This environment creates both a risk and an opportunity for PatchOps. The risk is that a vague thesis claim such as “we built an AI system that fixes CI” will sound derivative and underspecified. The opportunity is that most existing product narratives are not backed by public benchmarks, safety metrics, abstention analysis, taxonomy grounding, or reproducible academic evaluation. In other words, the concept is being commoditized, but the science is still underdeveloped.

PatchOps should therefore be understood as an academic response to an emerging product category. It is not trying to beat commercial tools at deployment scale or enterprise integration. It is trying to provide the measurement discipline, safety framing, and reproducibility that public product claims rarely supply.

2.10 Implications for PatchOps Design

Taken together, the literature suggests several design consequences for PatchOps.

First, log evidence extraction is necessary but insufficient. Systems such as LogSage show that careful log processing improves diagnosis, but CI-Repair-Bench and later PatchOps results show that diagnosis alone does not solve repair.

Second, repository-level reasoning is unavoidable. Real CI failures often require context beyond the visible log, including workflow files, manifests, and source modules. This supports PatchOps’ repository-aware design.

Third, validation must be embedded into the loop. The literature repeatedly shows that patch plausibility is a weak signal of correctness. PatchOps therefore uses CI-style re-execution as part of its decision logic rather than as an afterthought.

Fourth, abstention must be treated as a feature of competence, not as a failure of confidence. The taxonomy and flakiness literature make it clear that some failures should not be patched automatically. PatchOps therefore elevates rerun, preview, escalate, and abstain into first-class actions.

Fifth, safety and honesty checks are not optional. Overfitting and reward hacking are known risks in APR and LLM repair. PatchOps responds by explicitly measuring unsafe

actions, on-target behavior, and the difference between apparently green and genuinely trustworthy outcomes.

2.11 Chapter Conclusion and Positioning

The literature reviewed in this chapter leads to a clear and disciplined positioning for PatchOps. Prior work has already demonstrated that LLMs can assist with CI/CD diagnosis, that repository-aware CI repair can be benchmarked through workflow re-execution, and that software repair pipelines benefit from structured rather than purely free-form agent design. At the same time, prior work also makes clear that repair success remains low, that test- or CI-passing patches can still be wrong, and that current benchmarks under-measure the decision-theoretic aspects of safe automation.

PatchOps therefore does not claim novelty in patch generation alone. Its contribution lies in reframing CI repair as a *risk-sensitive, abstention-aware decision problem* built on top of repository context and closed-loop validation. That framing is supported, not contradicted, by the literature. The next chapter turns from this background to the formal problem definition, requirements, and scope of the PatchOps system.

CHAPTER 3

Problem Statement, Requirements, and Scope

Preface

This chapter formalizes the PatchOps problem, defines the action space and safety requirements, and fixes the practical scope of the thesis so later architecture and evaluation claims remain precise and defensible.

3.1 Introduction

The previous chapter established that CI repair is already a recognized research task, but that the most defensible remaining gap lies in *safe*, *abstention-aware*, and *operationally disciplined* automation. This chapter turns that positioning into a precise problem statement. Rather than treating PatchOps as a generic coding agent, the thesis defines it as a constrained decision system for failed GitHub Actions workflows. The system is designed to observe a failure, compress the relevant evidence, reason over repository context, and then choose an appropriate operational response under explicit risk constraints.

This precision matters for two reasons. First, it prevents overclaiming. A thesis that vaguely states “PatchOps fixes CI failures” would blur together diagnosis, patch generation, workflow validation, and safety. Second, a well-defined problem statement makes the evaluation meaningful. The benchmarks later in the thesis measure not only whether a patch can be produced, but whether the system chooses the right category of action for the right kind of failure.

3.2 Task Definition

PatchOps addresses the following task: given a failed GitHub Actions workflow execution and, when available, a local clone of the associated repository, determine the most appropriate response among a bounded set of operational actions and support that action with traceable evidence. The task is therefore not merely “generate a patch.” It is a structured decision problem over noisy evidence.

At a high level, the system input consists of:

1. the raw CI log or failure trace associated with a failed workflow run,
2. optional repository context, including workflow files, manifests, and likely fix files,
and

3. optional execution context used for downstream validation, such as a local reproducible runner.

The system output is a structured result containing compressed evidence, a failure classification, a root-cause summary, a final decision, and when appropriate a candidate patch and its safety assessment. In the current implementation this structure is explicit in the engine types: evidence, classification, root cause, patch, safety report, and decision are all materialized rather than left implicit in free-form text. This is an important design choice because it makes the system measurable and auditable.

3.3 Decision Space

The central abstraction in PatchOps is the *decision* rather than the patch. The system currently supports four explicit actions:

1. **Patch:** generate a patch that is eligible for automatic progression because the failure is judged low risk, confidence is sufficiently high, and the patch passes the safety and plausibility checks.
2. **Preview:** generate a candidate patch or repair suggestion, but keep a human in the loop rather than treating the change as ready for automatic action.
3. **Rerun:** recommend re-execution when the failure is likely transient or flaky rather than a stable code or configuration defect.
4. **Escalate:** decline automatic repair and forward the case for human review, typically because the failure is high risk, semantically unclear, or outside the safe editing envelope.

The broader thesis uses the language of *abstention*, and that remains conceptually correct. In PatchOps, abstention is realized operationally through the non-patch branches of the decision space. A rerun is an abstention from editing; an escalate decision is an abstention from automated repair; and preview is a human-in-the-loop abstaining mode for cases where a suggestion may be useful but an autonomous patch would be too strong. This interpretation keeps the thesis consistent with the literature while matching the implemented engine state machine exactly.

3.4 Failure Taxonomy and Risk Framing

PatchOps uses a fixed operational taxonomy of failure categories, because the action policy depends on the type of failure being handled. The current taxonomy contains ten core categories, with an **unknown** fallback for unresolved cases:

dependency_install_failure	unit_test_failure
runtime_version_mismatch	missing_system_dependency
workflow_yaml_error	permission_or_secret_failure
docker_build_failure	flaky_or_transient_failure
lint_format_failure	lockfile_mismatch

These categories are not arbitrary labels invented only for implementation convenience. They are the project’s operational abstraction over three sources: the broader GitHub Actions failure literature, the twelve-category structure of CI-Repair-Bench, and the practical needs of a decision policy that must distinguish repairable mechanical failures from risky or ambiguous ones. In later chapters, this taxonomy will be mapped explicitly to the literature-backed crosswalk used throughout the evaluation.

Each category is also assigned a risk class. In the present system, low-risk categories include formatting, version mismatches, lockfile issues, dependency installation issues, and workflow YAML fixes. Medium-risk categories include Docker build failures, unit test failures, and missing system dependencies. High-risk cases include permission and secret failures. Transient failures are handled separately through rerun logic, while unknown cases are treated conservatively. This risk structure is one of the mechanisms that turns PatchOps from a patch generator into a risk-aware decision system.

3.5 Confidence, Learned Decision Gate, and Intended Actions

The decision policy in PatchOps is not purely symbolic; it combines category-level risk with confidence-based decision logic. In the earlier implementation, the engine used two pinned thresholds: a lower confidence floor of 0.50 and a higher patch threshold of 0.75. These values made the policy reproducible, but they were still hand-set constants. The later Phase 15 refinement replaces the low-risk act-versus-hold test with an optional learned, isotonic-calibrated decision gate over pre-generation signals such as calibrated classifier confidence, category risk, flaky status, semantic ambiguity, log-named-file count, and evidence length.

Formally, let x_i denote the pre-generation feature vector for instance i , and let $x'_i = (x_i - \mu)/\sigma$ denote the standardized feature vector used by the decision model. The classifier-facing confidence signal used throughout the policy is

$$\text{conf}_i = \max_c \Pr(c \mid x_i) = \max_c \text{softmax}(W\phi(\text{evidence}_i))_c, \quad (3.1)$$

where c ranges over failure categories and $\phi(\cdot)$ denotes the evidence representation. On the low-risk lane, Phase 15 adds a learned act-versus-hold model:

$$\Pr(\text{act} \mid x_i) = \sigma(w^\top x'_i + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (3.2)$$

Because raw probabilities can be miscalibrated, PatchOps applies a non-decreasing isotonic calibrator $g(\cdot)$, fit on labeled act/hold outcomes:

$$g^\star = \arg \min_{g \text{ monotone}} \sum_i (g(s_i) - y_i)^2, \quad p_{i,\text{cal}} = g^\star(\text{Pr}(\text{act} \mid x_i)). \quad (3.3)$$

The final low-risk decision rule is then

$$d_i = \begin{cases} \text{ACT} & \text{if } p_{i,\text{cal}} \geq \tau, \\ \text{HOLD} & \text{if } p_{i,\text{cal}} < \tau, \end{cases} \quad \tau = 0.5. \quad (3.4)$$

In plain terms, the gate predicts whether a low-risk case should proceed toward an automatic patch or remain in preview mode.

The intended action is computed before patch generation. If a failure is classified as flaky or transient, the system chooses rerun rather than edit. If the category is high risk or the diagnosis confidence falls below the lower floor, the system escalates rather than guessing. Medium-risk categories default to preview. Low-risk categories become automatic patch candidates only when the confidence policy supports action. In the threshold baseline this means crossing the higher threshold; in the learned-gate version this means the calibrated decision model predicts act rather than hold. Otherwise the case remains preview-only.

The older hand-set rule can therefore be written explicitly as

$$\text{conf}_i \geq \text{TAU_HIGH} = 0.75. \quad (3.5)$$

This mathematical contrast makes the “remove the magic constant” point concrete: the learned gate replaces a single pinned threshold with a calibrated decision surface over multiple pre-generation signals. At the same time, the broader routing policy remains intentionally deterministic outside this low-risk lane: flaky cases still rerun, high-risk or unknown cases still escalate, and medium-risk cases still default to preview.

This design reflects the thesis claim that the cost of a wrong action is asymmetric. A false negative, where the system declines to patch a fixable low-risk issue, is inconvenient. A false positive, where the system opens or endorses an unsafe or semantically misguided patch, is often more damaging. The policy therefore intentionally biases the system toward caution.

3.6 Semantic Ambiguity and Human Judgement

One of the most important requirements in PatchOps is that not every technically detectable error should be treated as machine-fixable. Some failures require genuine human judgement about intended program behavior. The current implementation encodes

this idea through a semantic ambiguity backstop that catches signals such as type errors, assertion mismatches, and logic-oriented exceptions. Even if such a case appears low risk at the raw category level, the system downgrades an intended automatic patch to preview.

This requirement is conceptually important because it separates mechanical repair from semantic repair. A format or version mismatch usually has a narrow and observable fix space. A type mismatch, failed assertion, or logic exception may involve the intended behavior of the software, which is something the thesis does not claim to solve fully. By defining semantic ambiguity as a first-class boundary, PatchOps makes its limits explicit rather than leaving them implicit in later error analysis.

3.7 Safety Requirements

The core safety requirement of PatchOps is straightforward: the system must never treat patch generation as a license to edit arbitrarily. A candidate patch must pass deterministic safety checks before it can remain in the `patch` branch of the decision pipeline.

The current safety envelope has several parts:

1. **Forbidden-path protection.** Certain paths, such as environment files, secret-bearing locations, credentials, and private-key material, are never valid edit targets.
2. **Category-specific allow-lists.** For categories such as workflow YAML errors, runtime version mismatches, lockfile issues, and dependency installation failures, the patch must stay within a narrow set of allowed file patterns.
3. **Secret-introduction checks.** Added lines are scanned for patterns that resemble API keys, passwords, tokens, or private keys.
4. **Minimality constraints.** Automatic patches are bounded by maximum file count and maximum added-line count, reducing the chance of broad or unfocused edits.
5. **Reward-hacking detection.** The system checks for high-severity patterns such as test tampering, assertion removal, skip-based evasion, lint suppression, or CI-disabling changes that could make a run green without constituting an honest fix.

These requirements embody the thesis claim that safety must be engineered explicitly. They also ensure that PatchOps can be evaluated on more than repair success alone. Because the safety logic is encoded deterministically, the later unsafe-edit and abstention metrics are grounded in observable policy rather than vague intuition.

3.8 Problem Boundaries and Scope

The thesis scope is deliberately narrower than the full product fantasy of an autonomous DevOps engineer. The core comparable evaluation is Python-first because CI-Repair-Bench, the main public benchmark used throughout the project, is Python-only. Node and Docker examples remain useful in demonstrations and controlled cases, but they are not mixed into the main comparable repair table.

Similarly, PatchOps does not claim to emulate all of GitHub Actions locally. Its validation harness is a restricted CI-style re-execution loop meant for supported workflow shapes rather than a perfect clone of every GitHub-hosted runner feature. This limitation is not a weakness to hide. It is part of the problem definition: the thesis studies whether a safety-gated agent can make better decisions under a realistic but bounded validation regime.

The project also does not claim to solve semantic correctness in the strong sense. CI-green is treated as necessary but not sufficient evidence, and the system is explicitly designed to abstain, preview, or escalate when the evidence suggests that automated repair would be overconfident. This stance is consistent with both the APR overfitting literature and the measured behavior of PatchOps in later chapters.

3.9 Non-Goals

Several tempting claims are explicitly outside the scope of this thesis.

1. **State-of-the-art repair leadership** is not the primary claim. PatchOps is not framed as a system whose main contribution is beating the best published Pass@1 number.
2. **Universal workflow support** is not claimed. The thesis does not assert support for every language, every CI platform, or every GitHub Actions feature.
3. **Fully autonomous software engineering** is not claimed. PatchOps is a bounded CI-repair decision system, not a general autonomous development agent.
4. **Semantic correctness guarantees** are not claimed. A validated patch is stronger evidence than a free-form suggestion, but it is still not treated as a perfect proof of software correctness.
5. **Model-training novelty** is not the center of the thesis. Although the project includes a small learned classifier as a reproducibility and cost contribution, the main research focus remains orchestration, safety, abstention, and evaluation.

These non-goals are important because they protect the thesis from being judged against claims it does not need to make. A well-scoped thesis is stronger than an ambitious but indefensible one.

3.10 From Problem Statement to Architecture

The formal problem statement in this chapter now gives a clean lens for reading the rest of the thesis. PatchOps accepts failed CI evidence, reasons over bounded repository context, and chooses among patch, preview, rerun, and escalate actions under explicit risk and safety constraints. The system is therefore defined not by raw patch generation alone, but by a disciplined pipeline that links diagnosis, action selection, and validation.

This framing also clarifies why the architecture in the next chapter looks the way it does. The multi-stage pipeline, the safety gate, the semantic ambiguity backstop, the plausibility filter, and the validation loop are not independent engineering tricks. They are architectural consequences of the problem definition established here. The next chapter presents that architecture in detail.

3.11 Chapter Conclusion

This chapter has defined the PatchOps task, action space, failure taxonomy, decision policy, safety requirements, and scope boundaries. These definitions make the later architecture and evaluation measurable: PatchOps is judged not only by whether it can emit a patch, but by whether it selects an appropriate operational response under explicit risk constraints. The next chapter translates these requirements into the system architecture.

CHAPTER 4

System Design and Architecture

Preface

This chapter presents the PatchOps architecture, explains the role of each stage in the repair pipeline, and distinguishes the currently implemented single-pass workflow from the target agentic design used to motivate the thesis contribution.

4.1 Introduction

The previous chapter defined PatchOps as a risk-aware decision system for failed CI workflows. This chapter shows how that definition is realized architecturally. The most important high-level point is that PatchOps is not currently a free-form multi-agent swarm. It is a structured, multi-stage pipeline in which only some stages use LLM calls and the control flow itself remains explicit. This is a deliberate design decision, not a missing feature.

The architecture follows the evidence discussed in the literature review. Repair systems are easier to defend, measure, and ablate when the stages and transitions are visible. PatchOps therefore separates deterministic preprocessing, LLM-backed reasoning, deterministic policy logic, and downstream validation into distinct steps. This separation gives the thesis a clear systems story: repository-aware repair is not one monolithic prompt, but a controlled workflow that transforms raw CI evidence into an operational decision.

4.2 Architectural Philosophy

PatchOps is best understood as a *multi-stage decision pipeline* with an implemented validation harness around selected candidate patches. The core diagnosis-to-decision engine is single-pass, and the stronger multi-attempt verifier/rediagnosis loop remains the target architecture. The system already includes several LLM-backed components, but they are embedded in a deterministic state machine rather than being allowed to coordinate autonomously. In the terminology used throughout the project:

1. a **stage** is one step in the workflow,
2. a **deterministic stage** uses fixed rules or thresholds and does not call an LLM,

3. an **LLM-backed stage** performs bounded model-assisted reasoning, and
4. an **agentic loop** is a repeated act-observe-revise process in which the result of one action feeds back into another reasoning step.

This distinction is important because it clarifies what is already built and what is still proposed. The implemented PatchOps engine is a single pass from log to decision, and the project includes a local Docker validation harness for checking selected candidate fixes. The target architecture adds a tighter generate-validate-rediagnose loop and a verifier stage, which are the pieces that make the workflow genuinely agentic in the stronger sense.

4.3 Current Implemented Workflow

The current PatchOps workflow begins with a failed CI log and resolves to exactly one final action. The implemented stage order is:

1. log extraction,
2. failure classification,
3. flaky or transient detection,
4. repository context retrieval,
5. root-cause analysis,
6. patch planning and intended-action selection,
7. semantic ambiguity checking,
8. patch generation when a code change is warranted,
9. safety gating,
10. plausibility gating for candidate automatic patches, and
11. final decision emission.

This design directly matches the system objective established in Chapter 3. The pipeline does not attempt to “think about everything at once.” Instead, it narrows the decision space incrementally. Raw logs are first compressed into evidence. Evidence is then classified. The classification and extracted signals are used to decide whether the case is flaky, low risk, medium risk, high risk, or ambiguous. Only after these earlier filters does the system spend effort on candidate patch generation.

In the refined Phase 15 architecture, the low-risk act-versus-hold branch can also be routed through a learned decision gate. This gate does not replace the deterministic routing for flaky, high-risk, or unknown cases. Instead, it replaces the earlier hand-set high-confidence threshold on the low-risk lane with a calibrated model that predicts whether PatchOps should act or hold before generation.

4.4 Stage 1: Log Extraction

The first stage is a deterministic log extractor that performs the same kind of evidence compression suggested by the CI diagnosis literature. Its responsibilities are to remove formatting noise, identify the failed step, extract a critical error block, and detect known failure signatures. This stage is crucial because CI logs are far too noisy to be treated as raw model context in every case.

Architecturally, this stage serves two purposes. First, it reduces token cost and prompt clutter. Second, it creates structured evidence that can be reused by several later stages rather than forcing each one to reparse the original log independently. This is an example of design discipline that matters in systems research: expensive reasoning is deferred until the most relevant evidence has already been isolated.

4.5 Stage 2: Failure Classification

The classification stage maps the compressed failure evidence into the project’s operational taxonomy. In the current architecture, this stage is LLM-primary but can also incorporate rule hints and other supporting signals. The output is not only a category label but also a confidence value and rationale. That confidence value becomes a control signal for later policy decisions, which is why classification is not merely a reporting step.

This stage exists because subsequent decisions depend on failure type. A runtime version mismatch, a flaky network timeout, a missing secret, and a failing test should not be pushed through the same patching logic. Classification therefore acts as the bridge between raw diagnosis and risk-aware action selection.

4.6 Stage 3: Flaky and Transient Detection

Once the system has a compressed view of the failure, it checks whether the run looks flaky or transient. This stage is deterministic in the current implementation and is intentionally placed before repository reasoning and patch generation. The rationale is straightforward: if the observed failure is likely due to a network timeout, registry instability, connection reset, or similar transient issue, the safest response is rerun rather than repair.

This ordering is one of the clearest expressions of the thesis contribution. Many repair

systems ask “can I patch this?” before asking “should I patch this?” PatchOps flips that priority by giving transient handling an early exit branch in the architecture.

4.7 Stage 4: Repository Context Retrieval

If the failure is not classified as transient, PatchOps retrieves repository context. This stage is a key architectural differentiator from log-only diagnosis assistants. The retriever collects category-relevant files such as workflow YAML, manifests, configuration files, lockfiles, and evidence-linked source files. In the expanded full-repository mode described later in the thesis, it can also surface candidate fix files that the log never names directly.

The architectural role of this stage is to make the later reasoning repository-aware without letting repository size overwhelm the system. PatchOps does not dump the full repository into the model context. Instead, it uses a bounded retrieval step to select files that are likely to matter for the current failure. This is one of the main ways the system balances practicality with context sensitivity.

4.8 Stage 5: Root-Cause Analysis

The root-cause analysis stage is LLM-backed. It takes the extracted evidence, the classification result, and the retrieved repository files, and produces a summarized explanation of the likely defect and the affected files. Architecturally, this stage converts local evidence into a hypothesis about what actually went wrong.

This stage is distinct from classification for a reason. Classification says what type of failure occurred. Root-cause analysis says what in the repository most likely caused it. Keeping these two functions separate makes the system easier to analyze and avoids overloading a single prompt with too many tasks at once.

4.9 Stage 6: Patch Planning and Intended Action

After root-cause analysis, PatchOps computes an intended action through a deterministic planner. This stage combines failure category, confidence, and flaky status with the risk policy defined in Chapter 3. If the case is high risk or unknown, the planner escalates. If it is medium risk, the planner defaults to preview. If it is low risk and highly confident, the planner marks the case as a patch candidate. Otherwise, it remains preview-oriented.

This stage is architecturally central because it is where repair becomes explicitly decision-theoretic. PatchOps does not treat patch generation as the default outcome. Instead, it introduces a planner whose job is to decide whether patch generation is even appropriate to pursue strongly.

4.10 Stage 6b: Learned Decision Gate

The learned decision gate is the implemented answer to the question of where the act-versus-hold threshold comes from. Earlier versions of PatchOps used hand-set constants to decide whether a low-risk case was confident enough for automatic patching. Phase 15 replaces that low-risk threshold test with an optional calibrated model trained on the labeled Safety Set.

The gate uses only pre-generation signals: calibrated classifier confidence, category risk, flaky status, semantic ambiguity, the number of files named in the log, and evidence length. Its output is a calibrated probability of action. If the gate predicts act, the case may proceed toward patch generation and still must pass the safety and plausibility gates. If it predicts hold, the case remains preview-oriented. The rest of the policy is unchanged: flaky cases still rerun, high-risk and unknown cases still escalate, and low-plausibility or unsafe patches are still downgraded later.

Architecturally, this means Stage 6b implements the Chapter 3 rule $d_i = \text{ACT}$ iff $p_{i,\text{cal}} \geq 0.5$ on the low-risk lane only. It does not bypass the deterministic routing for flaky, medium-risk, high-risk, or unknown cases, and it does not overrule the later safety and plausibility gates. This separation matters because the learned model refines one bounded decision boundary inside the workflow rather than turning the entire system into an opaque end-to-end controller.

This stage matters architecturally because it turns abstention from a threshold convention into an evaluated model component. It also keeps the design conservative: the learned gate is opt-in and graceful. If the model artifact is absent, the earlier deterministic threshold behavior remains available.

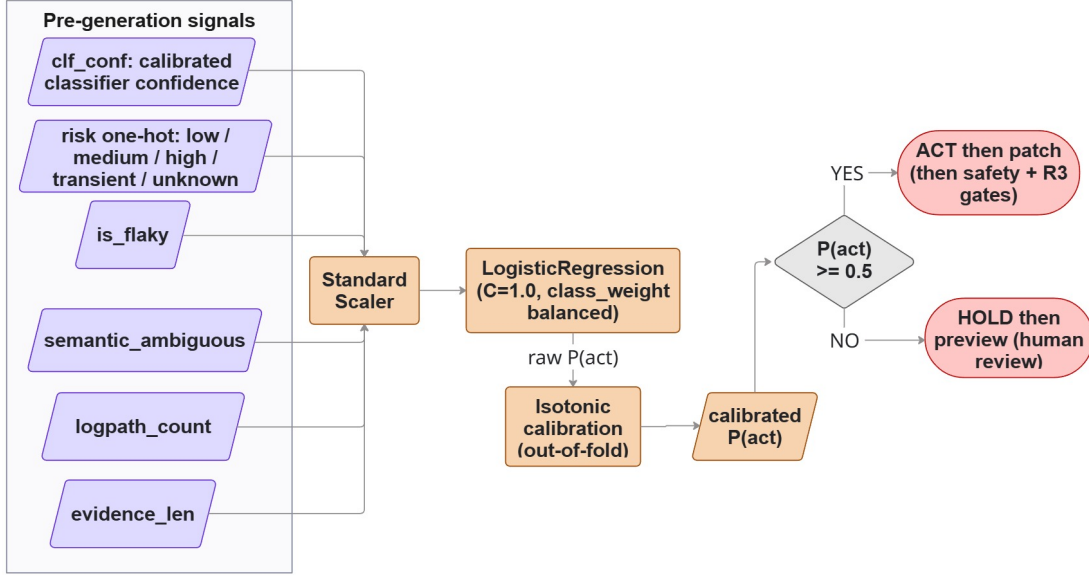


Figure 4.1: Learned decision gate architecture (Stage 6b). Deterministic, zero-cost pre-generation signals (calibrated classifier confidence, category-risk one-hot, flaky status, semantic ambiguity, log-named-file count, and evidence length) are standardized and passed to a calibrated logistic-regression model. The isotonic-calibrated probability of action p_{cal} is thresholded at 0.5 on the low-risk lane to set act (proceed to patch, then the safety and plausibility gates) versus hold (preview for human review).

4.11 Stage 7: Semantic Ambiguity Backstop

Before any candidate automatic patch can survive the pipeline, the system applies a semantic ambiguity check. This check is deterministic and looks for signals associated with type errors, assertion mismatches, and logic exceptions. If such signals are present, an intended automatic patch is downgraded to preview.

Architecturally, this backstop is important because it enforces a boundary between mechanical and semantic repair. It is also a good example of how PatchOps combines deterministic policy with LLM-driven reasoning: models may propose a plausible repair path, but a separate policy layer can still decide that the action should remain human-reviewed.

4.12 Stage 8: Patch Generation

When the planner and semantic guard still permit a code change, PatchOps generates a patch. This stage has both a deterministic rule path and an LLM path. The deterministic path handles flagship mechanical cases such as runtime version mismatches. The LLM path covers the broader set of repairable low- and medium-risk categories by producing candidate file edits and then deriving a unified diff from them.

This hybrid architecture is significant for the thesis. It shows that PatchOps is not a pure LLM wrapper. Where reliable deterministic repair is possible, the system prefers it.

Where broader reasoning is required, it uses an LLM under tightly constrained output expectations. The generated patch is accompanied by PR-facing metadata, including title and explanation, which supports the system’s live-review workflow.

4.13 Stage 9: Safety Gate

The safety gate is the core defensive component in the implemented architecture. It examines candidate patches before they are allowed to remain in the automatic patch branch. The gate checks for forbidden paths, category-specific allow-list violations, secret introduction, excessive patch size, and reward-hacking patterns such as test tampering or check-disabling edits.

If the gate blocks a patch, the system never proceeds as though the change were safe for automatic use. Depending on the case, the decision is downgraded to preview or escalate. This downgrade logic is architecturally important because it shows that safety is not a separate report generated after the fact. It actively changes the control flow of the system.

4.14 Stage 10: Plausibility Gate

Even a patch that passes the file-level safety gate may still be off-target. PatchOps therefore includes a plausibility gate that estimates whether the generated patch is likely to correspond to a real fix or whether it looks overfit. The plausibility score uses static signals such as evidence locality, minimality, category fit, source-touch behavior, deletion heaviness, and reward-hack absence.

This stage is the architectural response to a central finding of the APR literature: plausible does not imply correct. In PatchOps, a low-plausibility patch is not discarded silently. Instead, it is downgraded from automatic patch to preview. This preserves usefulness while protecting the system from overclaiming confidence in weak candidates.

4.15 Final Decision and Trace

After the previous stages have run, PatchOps emits one final decision. This decision, together with the underlying evidence, reasoning trace, patch, and safety report, is returned as a structured result. The trace-based design is intentional. Because the thesis emphasizes trustworthy automation, it is not enough for the system to merely output an action. The action should be inspectable, explainable, and attributable to intermediate stages.

This trace mechanism also supports the live demo interface and later analysis. A failed or downgraded patch is not a black box; the user can inspect whether the downgrade happened because of low confidence, a safety-policy violation, a semantic ambiguity

signal, or low plausibility.

4.16 Current Workflow as a Figure

For thesis presentation, the current implemented system should be shown as a single-pass architecture diagram. That figure starts from a CI failure log, passes through log extraction, classification, flaky detection, repository retrieval, root-cause analysis, planning, patch generation, safety checking, and decision emission, and ends in one of the four explicit actions. This is the correct “implemented pipeline” figure for the thesis because it corresponds exactly to the built system rather than to a purely aspirational design.

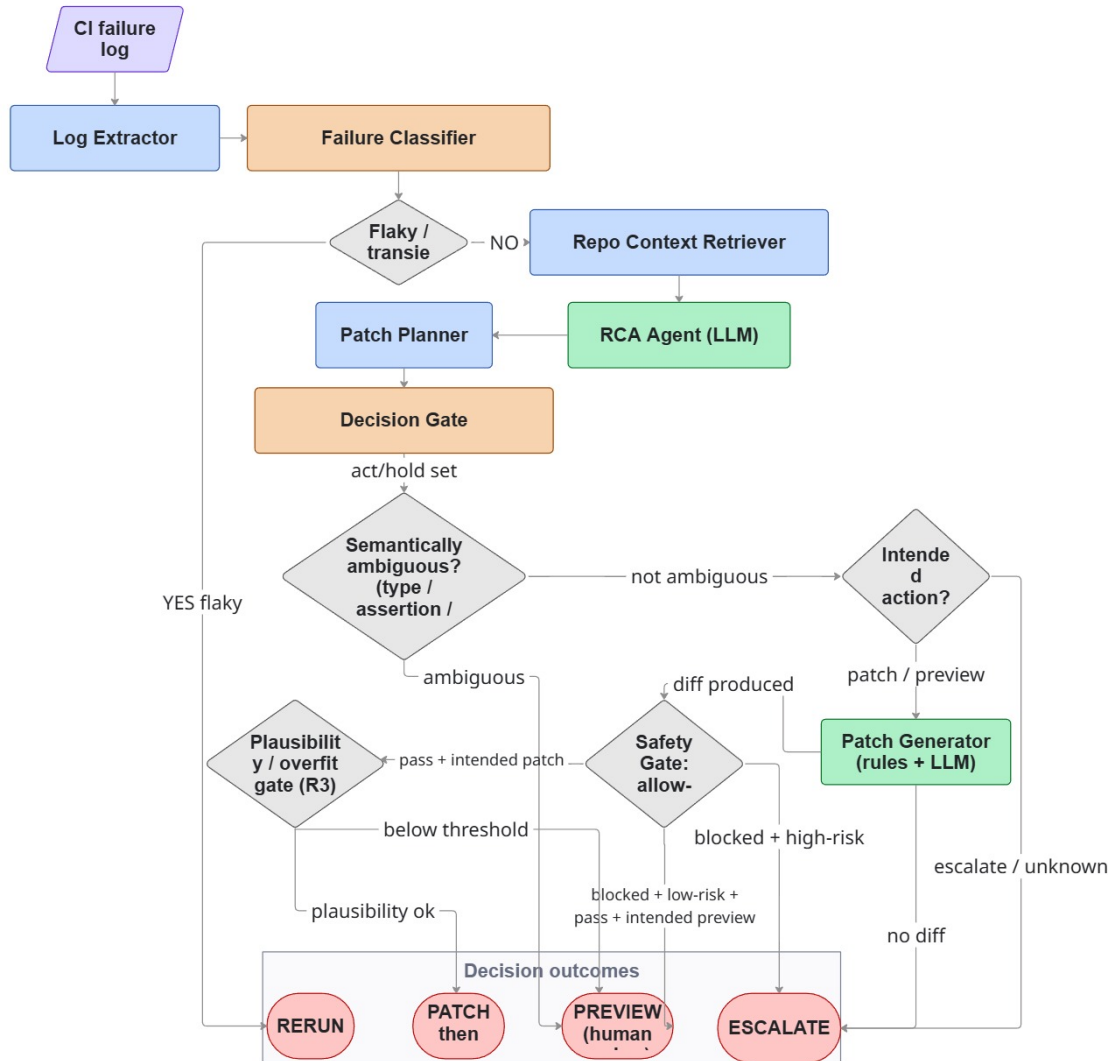


Figure 4.2: The implemented PatchOps pipeline end-to-end. A CI failure log is extracted and classified (TF-IDF + calibrated logistic regression, then LLM adjudication); flaky failures rerun, otherwise the system retrieves repository context, runs root-cause analysis, plans a risk class and intended action, and sets act/hold at the decision gate (learned calibrated gate on the low-risk lane, hand-tuned thresholds elsewhere). A semantic-ambiguity backstop, the safety gate, and the plausibility/overfit gate then route each case into exactly one of four explicit decision outcomes: rerun, patch (open PR), preview (human review), or escalate.

4.17 Validation Harness and Target Agentic Architecture

Although the core decision engine is single-pass, PatchOps already includes a bounded local validation harness used to apply candidate patches and re-run supported CI-style commands. This is the mechanism that makes CI-validated repair measurable in the thesis. The stronger target architecture tightens this mechanism into two additional capabilities: a verifier stage and a multi-attempt validation loop. These additions are what make the architecture genuinely agentic in the stronger sense described earlier.

The verifier stage is an additional reasoning layer that critiques a generated patch before it reaches the final safety and validation stages. Its purpose is to catch plausible-but-wrong fixes earlier. The validation loop extends the workflow from one-shot repair into generate-validate-rediagnose behavior: if a patch is applied and the CI-style validation still fails, the system re-enters diagnosis with the new error and attempts another repair within a bounded retry limit.

Architecturally, this target design matters because it gives the thesis a clear progression path. The current system is a carefully engineered decision pipeline with validation support. The target system is the same pipeline extended with the smallest additional agentic pieces that the literature suggests are worth the complexity. This keeps the design honest: agentic behavior is added where it promises measurable benefit, not for rhetorical effect.

4.18 Implemented Extensions and Target Growth

The broader architecture also accommodates two important extensions that were introduced after the earliest base pipeline and are now part of the implemented and evaluated PatchOps system: full-repository retrieval and a lightweight learned classifier. Full-repository retrieval expands the candidate search space beyond log-named files, addressing the localization bottleneck directly. The learned classifier supports reproducibility and low-cost classification while preserving the main thesis focus on orchestration and safety.

It is important to separate these implemented extensions from the still-targeted verifier and validation-loop additions described above. Full-repository retrieval and the learned classifier are no longer merely proposed; they have already been built and measured in later project phases. Their role in this chapter is architectural: they show that the pipeline is modular enough to accept stronger retrieval and classification components without changing the underlying action model or safety policy. This modularity is one of the strongest design properties of PatchOps as a research artifact.

4.19 Why the Architecture Matches the Thesis Claim

The PatchOps architecture is not merely a sequence of implementation modules. It is a direct structural expression of the thesis claim. The system begins by compressing evidence rather than trusting raw logs, retrieves bounded repository context rather than relying on generic code generation, separates diagnosis from action choice, and places explicit policy barriers between a candidate patch and a final automatic action. In doing so, it embodies four commitments:

1. **Repository awareness:** reasoning should be grounded in actual project files rather than logs alone.
2. **Safety gating:** candidate edits must satisfy deterministic policy checks before they are trusted.
3. **Abstention awareness:** the system should be able to choose non-patch actions as first-class outcomes.
4. **Closed-loop validation:** stronger versions of the system should observe the consequences of a patch through bounded re-execution rather than treating generation as the endpoint.

These commitments are what distinguish PatchOps from both log-only CI assistants and unconstrained coding-agent patterns.

4.20 Chapter Conclusion

This chapter described the PatchOps system design as a structured repair pipeline rather than a generic LLM application. The architecture begins with deterministic evidence compression, moves through LLM-assisted classification and root-cause analysis, and then narrows the decision through planning, the learned decision gate, semantic ambiguity checks, safety gating, plausibility assessment, and final action selection. The current engine is therefore a single-pass but decision-rich workflow, supported by a validation harness for selected candidate fixes.

The chapter also distinguished this implemented architecture from the target agentic design, which adds a verifier and a tighter bounded rediagnosis loop. That distinction is important for the thesis because it keeps the implemented system and the proposed research direction clearly separated. With the architecture now established, the next chapter can map these stages to the actual codebase and implementation details.

CHAPTER 5

Implementation

Preface

This chapter maps the PatchOps architecture to the codebase, showing how the repair pipeline is implemented in modular engine components, how validation and safety are enforced in practice, and how the system remains usable even when optional LLM or learned-model layers are unavailable.

5.1 Introduction

The previous chapter described the PatchOps architecture as a structured repair pipeline. This chapter maps that design onto the actual implementation. The goal is not to list every file mechanically, but to explain how the main modules cooperate to realize repository-aware, safety-gated, abstention-aware CI repair in code.

The implementation centers on the `patchops.engine` package, with supporting validation, adapter, and benchmark code around it. This separation is intentional. The engine handles reasoning and decision logic. The validation layer handles CI-style re-execution. The benchmark scripts use the engine as a reusable subject of study rather than duplicating repair logic outside the system.

5.2 Implementation Structure

The PatchOps implementation is organized around a small set of coherent modules rather than a monolithic agent script. At the highest level, the codebase is divided into:

1. **engine modules** under `patchops/engine/`, which implement diagnosis, retrieval, planning, generation, safety, and final decision logic,
2. **validation modules** under `patchops/validation/`, which provide local CI-style re-execution,
3. **adapters** under `patchops/adapters/`, which support GitHub- and CLI-facing integration,
4. **benchmark scripts** under `benchmarks/`, which evaluate the engine under controlled experimental protocols, and

5. **acceptance and verification tests** under `tests/`, which enforce regression guarantees on the core behavior.

This layered structure is valuable for a thesis artifact because it keeps the experimental system reusable. The same `diagnose(...)` entry point used in the live or local workflow can also be exercised by the benchmark harness, making the evaluation more faithful to the actual system behavior.

5.3 The Engine Entry Point

The practical center of the implementation is the decision routine in `patchops/engine/decide.py`. This module realizes the state machine described earlier: log extraction, classification, flaky detection, context retrieval, root-cause analysis, intended-action planning, semantic ambiguity downgrading, patch generation, safety gating, plausibility scoring, and final decision selection.

The implementation returns a structured `Result` object rather than plain text. This object bundles the evidence, classification, root cause, decision, optional patch, optional safety report, and the reasoning trace. Supporting dataclasses in `types.py` make the pipeline explicit and serialization-friendly. Architecturally, this means every stage writes to a shared, typed interface rather than leaking unstructured prompt outputs across the system.

5.4 Evidence Compression and Classification

The first implementation layer is evidence compression. The log extractor in `patchops/engine/log_extractor.py` performs deterministic preprocessing: it strips ANSI noise, removes timestamp clutter, detects known signatures, finds the failed step, and isolates a critical error window. This keeps the later prompts short and makes the engine usable even without a live model.

Classification is implemented in `classifier.py`. Its policy is hybrid. Strong deterministic signatures remain useful, but noisy real logs often require model adjudication. When an LLM is available, the classifier uses the rule-based hint and the learned-classifier hint as inputs to an LLM-primary decision. When no LLM is available, a confident learned-classifier prediction becomes the offline primary; otherwise rule signatures are used as the fallback. This preserves the thesis claim that PatchOps degrades gracefully rather than collapsing without API access.

5.5 Provider-Agnostic LLM Layer

One of the most important implementation choices is the provider-agnostic LLM abstraction in `patchops/engine/llm.py`. Instead of hard-wiring the system to one

vendor, PatchOps supports OpenAI, Anthropic, and OpenAI-compatible providers behind a shared interface. The implementation also supports local or alternative OpenAI-style endpoints through a configurable base URL. If no provider is configured, the engine returns cleanly to deterministic behavior.

This design serves three purposes. First, it makes experiments reproducible by pinning model roles through configuration. Second, it lets the system distinguish between strong and cheaper models for different stages, such as classification versus patch generation. Third, it turns LLM access into an optional acceleration layer rather than a hard dependency. That is especially important for a thesis claiming reproducibility and careful systems design rather than merely cloud-backed prompt orchestration.

5.6 Repository Context and Localization

Repository awareness is implemented through a pair of cooperating modules: `context_retriever.py` and `localizer.py`, with `repo_index.py` extending the design for full-repository search. The context retriever collects category-relevant files such as workflow YAML, manifests, lockfiles, configuration files, and evidence-named source files. It does so under size caps to keep prompt cost bounded.

The localizer makes file selection an explicit, measurable subproblem. It implements multiple strategies, including log-path ranking, category-aware reranking, LLM candidate generation, hybrid fusion, deterministic full-repository BM25 ranking, and semantic embedding reranking. The repository index in `repo_index.py` builds a candidate universe from the repository clone itself, applies path-aware BM25, and optionally reranks the deterministic pool. This modular structure is important because it lets the thesis study localization as its own bottleneck rather than treating file selection as an invisible side effect of a single large prompt.

5.7 Root-Cause Analysis and Patch Planning

Once context has been gathered, PatchOps performs root-cause analysis through `rca_agent.py`. This module combines rule-based templates with optional LLM refinement. The implementation reflects the thesis philosophy of bounded intelligence: rules provide a defensible baseline and zero-cost fallback, while the model can produce a more contextual summary and affected-file hypothesis when available.

The planner in `patch_planner.py` then turns diagnosis into intended action. It maps category, confidence, and flakiness into the action policy described earlier, using pinned thresholds and fixed risk classes from `config.py`. The semantic ambiguity guard is also implemented here, allowing the engine to downgrade an intended patch to preview when type-checking or logic-oriented signals suggest that human judgement is still required.

The learned decision gate is implemented in `decision_gate.py` and wired into `decide.py`. It is opt-in and replaces only the low-risk high-confidence act-versus-hold test when the environment enables it and the model artifact is available. It does not bypass flaky rerun routing, high-risk escalation, the safety gate, or the plausibility gate. This narrow integration is deliberate: the learned gate removes the old magic threshold on the low-risk lane while preserving deterministic safety boundaries.

5.8 Patch Generation

Patch generation is implemented in `patch_generator.py`. The module contains both deterministic repair paths and LLM-backed patch generation. The deterministic path handles important mechanical cases such as runtime version mismatches, where the repair can be computed directly from repository and log evidence. The LLM path handles a broader set of repairable categories by asking the model to return full edited file contents and then computing the unified diff locally.

This is an important implementation detail. Asking the model for raw diff text directly is brittle. By asking for full edited file contents and deriving the diff in code, PatchOps keeps patch formation more controllable and more likely to remain `git apply`-compatible. The generator also produces review-oriented PR metadata, including a title and a security-conscious PR body, reinforcing that the output is meant for inspection rather than blind execution.

5.9 Safety and Plausibility Controls

The two most thesis-critical safety modules are `safety_gate.py` and `patch_plausibility.py`. The safety gate implements deterministic checks over the candidate patch. It blocks forbidden paths, category-level allow-list violations, secret introduction, excessive patch breadth, and reward-hacking patterns. Because this logic is deterministic and explicit, it can be measured directly in the evaluation rather than inferred informally.

The plausibility module implements a second layer of control. It estimates whether a generated patch is likely to be on-target using only inference-time signals such as evidence locality, file-type fit, minimality, and honesty. Low-plausibility patches are downgraded from automatic patch to preview. Together, these two modules operationalize the thesis argument that CI-green and patch-generated are not strong enough success conditions on their own.

5.10 Learned Classifier Integration

The learned classifier is implemented as an additive layer in `ml_classifier.py`. It loads a trained artifact produced by the benchmark training script and predicts category

plus confidence from the same compressed evidence representation used at inference time. If the artifact is missing or the supporting dependencies are unavailable, the module simply yields no prediction and the engine falls back to its earlier rule and LLM path.

This is an important implementation property. The classifier does not destabilize the rest of the system. It is integrated as a graceful enhancement for reproducibility and low-cost inference, not as a mandatory dependency that would change the rest of the engine’s control flow.

The classifier artifact also carries the shipped isotonic calibrator used by later decision stages. When present, `ml_classifier.py` applies this calibrator before emitting confidence; when absent, it falls back to raw confidence. This makes calibrated confidence an inference-time engine property rather than only an offline plotting result.

5.11 Validation Harness

The CI validation layer is implemented under `patchops/validation/`. Its local runner parses common GitHub Actions workflow shapes, extracts runnable shell steps, and executes them either in Docker or on the host as a fallback. The implementation is deliberately modest about scope: it supports common `setup-node` and `setup-python` patterns plus shell-based workflow steps, but it is not a full GitHub Actions emulator.

This bounded implementation is nevertheless valuable. It gives PatchOps a local, repeatable mechanism for testing whether a candidate patch changes the observable CI outcome. The validation layer is therefore what connects the abstract notion of repair to the measurable Pass@1 and Pass@3 style metrics used later in the thesis.

5.12 Benchmarks as First-Class Implementation Clients

The benchmark scripts are not bolted on as an afterthought. They are designed as disciplined clients of the same engine. For example, `benchmarks/run_eval.py` drives the diagnosis and patch pipeline across ablation variants, `run_localization.py` isolates file-selection strategies, and the later benchmark scripts evaluate full-repository retrieval, the learned classifier, selective prediction, plausibility, reward hacking, agentic localization, and the learned decision gate.

This matters because it means the measured system is the real system. The thesis is not evaluating simplified mock-ups of PatchOps stages; it is evaluating the actual implemented modules under controlled harnesses.

5.13 Acceptance and Regression Tests

PatchOps also includes acceptance and regression tests that enforce the system’s core contract. The acceptance harness checks end-to-end diagnosis on the ten-case example

set, verifies that the flagship deterministic patch applies cleanly, confirms correct rerun and escalate behavior on flaky and secret-related examples, requires a reasoning trace, and checks both the semantic ambiguity and reward-hacking protections.

These tests are important in a thesis context because they distinguish two levels of confidence. Experimental results tell us how the system behaves on a benchmark. Acceptance checks tell us whether the intended design properties still hold after implementation changes. Together, they strengthen the system as a research artifact.

5.14 Implementation Principles

Several implementation principles recur across the codebase:

1. **Graceful degradation.** Optional LLM, classifier, retrieval, and decision-gate enhancements are additive rather than destructive; the system retains deterministic functionality when they are unavailable.
2. **Bounded cost.** File size caps, prompt truncation, top-k limits, and staged retrieval keep model usage and runtime manageable.
3. **Deterministic policy barriers.** Safety, ambiguity, and risk checks are enforced in code rather than left to prompt compliance.
4. **Modularity.** Retrieval, classification, generation, safety, and validation are separable enough to be ablated experimentally.
5. **Traceability.** Each stage contributes explicit outputs to a reasoning trace that can be inspected later by users or benchmark harnesses.

These principles are what make PatchOps more than a prompt chain. They turn it into a controlled software system whose behavior can be studied systematically.

5.15 Chapter Conclusion

This chapter mapped the PatchOps architecture to its concrete implementation. The engine centers on a typed, traceable decision pipeline; the LLM layer is provider-agnostic and optional; repository retrieval and localization are modular and measurable; patch generation combines deterministic and model-backed paths; the learned decision gate removes the old low-risk magic threshold when enabled; safety and plausibility are enforced through explicit gates; and the validation harness provides bounded CI-style re-execution. The benchmark scripts and acceptance tests then exercise the same engine as both an experimental subject and a usable tool.

With the implementation now grounded in code, the next chapter can describe how the system was evaluated: which datasets were used, how baselines were defined, what

metrics were frozen in advance, and how the project maintained research discipline through phase-by-phase pre-registration.

CHAPTER 6

Experimental Methodology

Preface

This chapter explains how PatchOps was evaluated, including the dataset strategy, baseline ladder, metric definitions, validation coverage rules, and the pre-registration discipline used to keep claims honest across the project phases.

6.1 Introduction

PatchOps is a systems thesis, but it is also an evaluation thesis. The contribution depends not only on whether the software works in isolated examples, but on whether its behavior can be measured honestly against baselines on defined datasets. This chapter therefore explains the evaluation methodology used throughout the project.

The methodology reflects a deliberate response to advisor feedback and to the risks identified in the literature review. CI repair is easy to overclaim and hard to benchmark cleanly. Public benchmarks emphasize repair success, but the main PatchOps claim concerns safety, abstention, and decision quality. Workflow re-execution is expensive and imperfectly reproducible. Some important failure types, especially secret, permission, and transient cases, are under-represented in public repair benchmarks. For these reasons, PatchOps uses a multi-source evaluation strategy and an explicit phase-by-phase pre-registration workflow.

6.2 Evaluation Philosophy

The first methodological principle in PatchOps is that a working component is not automatically a result. Implementation progress and experimental evidence are treated separately. A component counts as a result only when it is measured against a baseline on a clearly defined dataset under frozen metric definitions.

The second principle is that every reported number must carry its context. Sample size, coverage, and limitations are not optional details added after the fact. They are part of the result itself. This principle is especially important in a thesis where some sub-studies use modest sample sizes or only subsets of workflows that can be validated locally.

The third principle is that negative findings remain findings. PatchOps explicitly preserves honest negative results such as retrieval plateaus or weak agentic gains.

Methodologically, this matters because it reduces the temptation to tune or rewrite the experimental story around only the favorable cells.

6.3 Phase-by-Phase Pre-Registration

One of the strongest research-discipline choices in the project is the use of a repeated pre-registration-like workflow for each major evaluation phase. Every significant experimental phase is documented with four files:

1. `benchmark.md`, written before execution, freezing the research question, dataset, baselines, metric definitions, and hypotheses;
2. `methodology.md`, written before execution, describing the harness, scoring rules, and known threats to validity;
3. `findings.md`, written after the run, recording the observed numbers; and
4. `conclusion.md`, written after the run, interpreting the findings.

This workflow does not claim the formal status of clinical-style pre-registration, but it serves the same practical purpose for a thesis project: it constrains post-hoc story drift. An especially important rule follows from this structure: when a number conflicts across summary documents, the per-phase `findings.md` is the canonical source.

6.4 Datasets and Evaluation Sources

PatchOps does not rely on a single benchmark for every research question. Instead, it uses different datasets for different forms of evidence.

The primary public benchmark for repair and diagnosis is **CI-Repair-Bench**, a Python-first benchmark of 567 CI failure instances distributed across more than one hundred repositories and twelve categories. This is the anchor dataset for comparable repair performance, localization, and diagnosis experiments [2].

For broader CI log context and classifier motivation, PatchOps also references larger CI log corpora such as **GHALogs**. These corpora are useful for understanding real-world failure distributions and classification context, even when they are not directly suitable for patch validation [9].

For safety and abstention, PatchOps uses a curated **Safety Set**. This is necessary because public repair benchmarks under-sample the exact categories that matter most to the thesis claim, such as permission failures, missing secrets, deployment-sensitive cases, and clearly transient failures where rerun is preferable to repair. The Safety Set first appears as a 20-case labeled evaluation in Phase 5 and is later expanded to 76

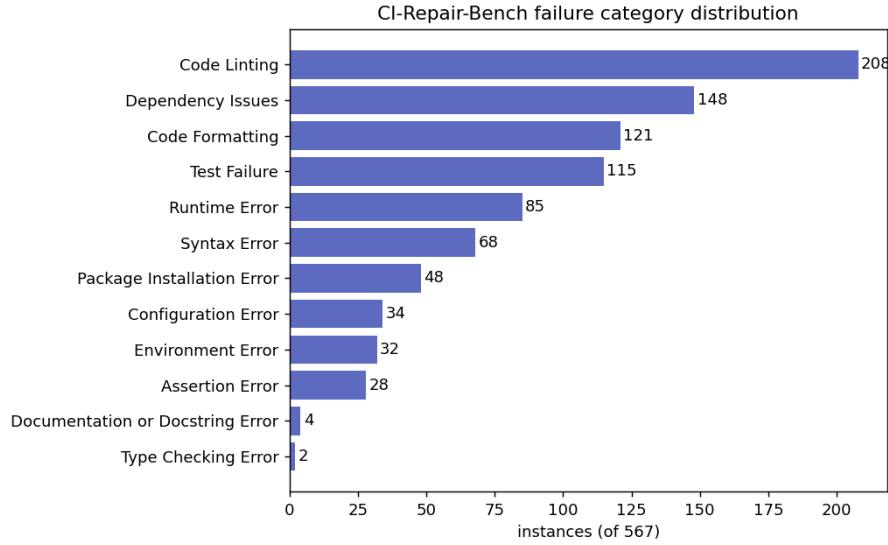


Figure 6.1: CI-Repair-Bench category distribution used as the main public benchmark context.

labeled cases in Phase 15 for the learned decision-gate study. The two uses are reported separately so the original Phase 5 safety result is not rewritten after the fact.

Finally, live demonstration cases exist separately from benchmark evidence. These cases are useful for viva or system demonstration but are not mixed into the research tables.

6.5 Why a Multi-Source Design Is Necessary

This multi-source design is not a convenience; it is methodologically necessary. A single public benchmark is well-suited for repair comparability but poorly suited for safety or abstention claims when risky classes are rare. Conversely, a fully synthetic benchmark may provide excellent safety ground truth while saying little about real repair difficulty. By combining a public repair benchmark with a labeled safety benchmark and a separate demo set, PatchOps aligns each research question with the kind of evidence it actually needs.

This also helps separate benchmark scope from product scope. The comparable core thesis result remains Python-first through CI-Repair-Bench. Broader language examples may still appear in controlled cases or live demonstrations without weakening the comparability of the main evaluation.

6.6 Research Questions and Evaluation Axes

The experimental design is organized around fixed research questions rather than around arbitrary benchmark runs. Across the project phases, the questions remain stable:

1. whether repository context improves diagnosis and localization,

2. whether PatchOps improves patch quality or repair success relative to simpler baselines,
3. whether safety-gated decision making reduces unsafe or inappropriate repair actions,
4. whether PatchOps can distinguish repairable failures from flaky or transient ones, and
5. what cost and latency profile the system exhibits in realistic use.

Later phases add sharper sub-questions, such as whether confidence is a usable abstention signal and whether agentic navigation improves localization. However, these are extensions of the same core evaluation frame, not separate and unrelated experiments.

6.7 Baselines and Ablation Ladder

PatchOps is evaluated not only as a single full system, but also through an ablation ladder. This is essential because the thesis claims modular value from log preprocessing, workflow context, repository retrieval, safety gating, and selective decision logic.

The baseline ladder includes progressively richer context conditions such as raw or cleaned logs, workflow-aware reasoning, repository-aware reasoning, a full pipeline without the safety gate, and the full PatchOps pipeline. This structure allows the evaluation to answer questions such as whether workflow context improves classification, whether repository retrieval helps localization, and whether the safety gate or abstention logic meaningfully changes behavior relative to a raw generator tendency.

The same ladder logic also appears in specialized studies. Localization experiments compare multiple retrieval strategies on the same instances. Phase 8 extends this with full-repository BM25 retrieval. Phase 10 adds semantic embedding reranking. Phase 14 adds an agentic localizer. Together these comparisons produce a multi-method study of the localization bottleneck rather than a single isolated claim.

6.8 Validation Strategy

PatchOps uses a bounded local CI-style validation harness rather than claiming full GitHub Actions emulation. This distinction is methodologically important. The local runner parses common workflow shapes, extracts runnable steps, and executes them in Docker where possible. This allows reproducible patch validation for supported workflows while keeping the scope honest.

Because not every workflow can be replayed locally under the same assumptions, validation coverage is always reported explicitly. A workflow that cannot be supported by the local harness is not silently treated as a pass or hidden from the denominator

without disclosure. This coverage discipline is necessary to keep Pass@1 and related repair metrics interpretable.

6.9 Metric Families

The methodology tracks several families of metrics rather than collapsing everything into one success rate.

Diagnosis metrics include category accuracy and, where relevant, root-cause-related behavior. These measure how well the system identifies the type of failure before attempting repair.

Localization metrics include $\text{Recall}@ \{1,3,5\}$ and $\text{Hit}@ \{1,3,5\}$, comparing ranked predicted files against the files touched by the developer’s actual fix under a frozen matching rule. These metrics are important because localization is a known bottleneck and is studied explicitly rather than buried inside repair.

Patch metrics include patch-produced rate, patch-apply rate, and CI validation outcomes such as Pass@1 and Pass@3 where applicable. The methodology keeps patch-apply distinct from repair success, because a syntactically applicable diff may still be semantically useless.

Decision metrics are especially important for PatchOps. They include decision accuracy, abstention accuracy, unnecessary-patch rate, correct rerun behavior, and correct escalation behavior. These are the metrics that make the safety-and-abstention claim measurable.

Safety metrics include unsafe auto-fix rate and, in later refinements, separation between unsafe automatic actions and unsafe suggestions. These metrics operationalize the difference between emitting a risky idea for human inspection and automatically treating it as a deployable fix.

Calibration and selective-prediction metrics are added in the selective prediction phase. These include risk-coverage curves, area under the risk-coverage curve, reliability diagrams, and expected calibration error. Their purpose is to turn “abstain” from a slogan into a quantitative operating characteristic.

Practicality metrics include token cost, model usage, and latency, because a CI assistant that is too slow, too expensive, or too brittle to run repeatedly is not operationally credible.

Table 6.1: Metric families used in the PatchOps evaluation.

Metric family	Representative metrics
Diagnosis	Category accuracy, per-category accuracy, classifier confidence.
Localization	Recall@1/3/5, Hit@1/3/5, coverage, localization cost.
Patch and repair	Patch-produced rate, patch-apply rate with coverage, CI Pass@1/Pass@3.
Decision quality	Patch/preview/rerun/escalate accuracy, abstention accuracy, unnecessary-patch rate.
Safety and honesty	Unsafe auto-fix rate, reward-hack rate, patch plausibility, on-target precision.
Calibration	AURC, reliability, ECE, selective accuracy at fixed coverage.
Practicality	Token cost, wall-clock latency, model/provider dependence.

6.10 Formal Metric Definitions

To keep the later results precise, the metric families above are defined formally once here and then reused throughout Chapter 7. Let N denote the number of instances, let $\mathbf{1}[\cdot]$ denote the indicator function, let G_i denote the gold fix-file set for instance i , and let P_i^k denote the top- k predicted files returned by the localizer.

6.10.1 Localization Metrics

PatchOps reports both overlap-sensitive Recall@ k and hit-style Hit@ k :

$$\text{Recall@}k = \frac{1}{N} \sum_{i=1}^N \frac{|G_i \cap P_i^k|}{|G_i|}, \quad \text{Hit@}k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[G_i \cap P_i^k \neq \emptyset]. \quad (6.1)$$

Recall@ k measures how much of the gold fix set is recovered on average, while Hit@ k asks the simpler question of whether at least one correct file appears in the top- k list.

6.10.2 Repair Metrics

Repair metrics are always interpreted together with coverage. Let N_{att} denote the number of instances for which PatchOps actually attempted a patch under the given policy:

$$\text{Pass@}1 = \frac{1}{N_{\text{att}}} \sum_i \mathbf{1}[\text{patch}_i \text{ turns CI green}], \quad (6.2)$$

$$\text{apply-rate} = \frac{\#\{\text{patches that git apply}\}}{N_{\text{att}}}, \quad \text{coverage} = \frac{N_{\text{att}}}{N}. \quad (6.3)$$

This makes the honesty constraint explicit: a selective system can appear stronger on attempted cases partly by declining the hard ones, so repair quality should never be read without its corresponding coverage term.

6.10.3 Classification Metrics

For classification studies, PatchOps reports per-class precision and recall through the standard F_1 family. For class c ,

$$F_{1,c} = \frac{2P_c R_c}{P_c + R_c}, \quad \text{macro-}F_1 = \frac{1}{C} \sum_{c=1}^C F_{1,c}, \quad (6.4)$$

where C is the number of classes. Macro- F_1 is useful here because weak categories should remain visible instead of being dominated by larger easier classes.

6.10.4 Decision and Abstention Metrics

PatchOps is evaluated as a decision system, not only as a patch generator. If $\hat{a}_i$ is the predicted action and a_i^* is the expected action label, act/hold accuracy is

$$\text{Acc}_{\text{act/hold}} = \frac{1}{N} \sum_i \mathbf{1}[\hat{a}_i = a_i^*]. \quad (6.5)$$

Abstention accuracy uses the project's restricted denominator:

$$\text{Acc}_{\text{abst}} = \frac{\#\{\text{should-abstain} \wedge \text{did abstain}\}}{\#\{\text{should-abstain}\}}. \quad (6.6)$$

This definition matters because abstention is judged only on the subset where abstaining was the correct behavior; it is not averaged over all decisions.

6.10.5 Selective Prediction and Calibration

For a confidence score s_i , threshold τ , and zero-one loss $\ell_i \in \{0, 1\}$, selective prediction is summarized through coverage, selective risk, and area under the risk-coverage curve:

$$\text{cov}(\tau) = \frac{1}{N} \sum_i \mathbf{1}[s_i \geq \tau], \quad \text{risk}(\tau) = \frac{\sum_i \ell_i \mathbf{1}[s_i \geq \tau]}{\sum_i \mathbf{1}[s_i \geq \tau]}. \quad (6.7)$$

$$\text{AURC} = \frac{1}{N} \sum_{k=1}^N \left(\frac{1}{k} \sum_{j=1}^k \ell_{(j)} \right), \quad (6.8)$$

where $\ell_{(j)}$ are losses sorted by decreasing confidence. Lower AURC indicates a better confidence ranking because the accepted prefix stays more accurate over a wider range

of coverage values.

Calibration is reported through expected calibration error. For M confidence bins B_m ,

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|. \quad (6.9)$$

ECE measures whether stated confidence matches empirical correctness, which is why it is central to the abstention story in Phase 11 and the learned decision gate in Phase 15.

6.10.6 Plausibility and Ranking Quality

Patch plausibility is evaluated through ROC AUC, which can be read as the probability that a random on-target patch receives a higher score than a random off-target patch:

$$\text{AUC} = \frac{1}{|\mathcal{P}||\mathcal{N}|} \sum_{p \in \mathcal{P}} \sum_{n \in \mathcal{N}} \mathbf{1}[s_p > s_n]. \quad (6.10)$$

This matches the role of the plausibility gate as a ranking mechanism rather than as a proof of semantic correctness.

6.10.7 Confidence Intervals

Because several PatchOps studies operate at small n and sometimes near 0 or 1, Wilson score intervals are preferable to normal approximations. For an empirical proportion $\hat{p}$ and $z = 1.96$,

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}. \quad (6.11)$$

This is the interval family behind the thesis wording that Phase 15 matches-or-beats the threshold baseline rather than claiming a clean statistical separation at $n = 76$.

6.11 Ground-Truth Design for Safety and Flakiness

Safety and flakiness require more than a public benchmark label. The Safety Set therefore assigns structured labels to each case, including expected action, risk class, allowed files, forbidden files, category, and whether the case is flaky or transient. This allows PatchOps to be evaluated on the action it should have taken, not merely on whether it happened to emit a patch.

This is methodologically important because the main PatchOps claim is asymmetric. On some failures, a patch is the correct action. On others, not patching is the correct action. A benchmark that cannot represent that distinction cannot evaluate the thesis contribution properly.

6.12 Localization Methodology as a Separate Study

The project treats localization as an explicit experimental subproblem. Rather than assuming that retrieved context is always adequate, PatchOps compares log-path ranking, category reranking, LLM-guided candidates, hybrid ranking, full-repository BM25, semantic embedding reranking, and later agentic exploration. All of these studies reuse the same benchmark family and matching discipline, which makes it possible to interpret later findings as a single evolving localization study rather than disconnected experiments.

This explicit localization design is one of the strongest methodological choices in the thesis. It enables later claims such as “localization is necessary but insufficient” to be supported by measured transitions rather than by informal intuition.

6.13 Classifier Methodology as a Reproducibility Study

The learned classifier phase is methodologically scoped as a reproducibility and cost study rather than as a raw repair-improvement claim. The classifier is trained on compressed evidence features derived from the same preprocessing pipeline the live engine uses. It is evaluated on held-out data under both exact and lenient labeling rules and is compared with rule-based and LLM-based classification behavior.

This framing matters because it keeps the classifier’s role proportional. It is not presented as the central intelligence of PatchOps, but as a controlled and measurable way to make one stage of the pipeline open, cheaper, and more reproducible.

6.14 Selective Prediction and Calibration Methodology

The selective prediction phase extends the evaluation from action accuracy to confidence quality. Here, the question is whether the system’s confidence signals can support principled abstention. The methodology compares learned-classifier confidence against engine decision confidence on held-out data and evaluates whether acting on only the more confident subset improves accuracy. Calibration methods such as isotonic regression are used to measure whether these confidence values can be made more reliable without changing their ranking behavior.

This is important because it turns the thesis claim about abstention into something operationally tunable. A system can then be evaluated not only at one fixed threshold, but along a family of possible operating points.

6.15 Learned Decision-Gate Methodology

The Phase 15 decision-gate study extends selective prediction from confidence analysis into the actual act-versus-hold policy. The gate is trained and evaluated on the expanded

76-case Safety Set using cross-validation. Its inputs are pre-generation signals that the engine already computes, including calibrated classifier confidence, category risk, flaky status, semantic ambiguity, log-named-file count, and evidence length. The comparison baseline is the earlier hand-tuned threshold policy.

This methodology is intentionally scoped as a decision-quality evaluation, not a repair-success evaluation. The reported metrics are act/hold accuracy, abstention accuracy, over-act count, calibration error for $P(\text{act})$, and risk-coverage behavior. The study also reports the low-risk lane separately, because that is the branch where the deployed engine actually uses the learned gate.

Two caveats are built into the methodology. First, the expanded Safety Set is synthetic-heavy, so the result is an indicative labeled-set result rather than a field measurement. Second, at $n = 76$, confidence intervals overlap with the hand-tuned policy. The correct claim is therefore that the learned gate matches-or-beats the threshold policy while removing the magic constants, not that it is statistically separated.

6.16 Agentic Localization Methodology

The agentic localization phase adds a bounded read-only navigation loop over a repository clone. This phase is important methodologically because it tests whether the localization bottleneck is due merely to weak retrieval or whether even tool-using exploration struggles to recover the right files. The experiment keeps the slice, matching rules, and baseline comparisons aligned with earlier localization phases so that any gain or non-gain is interpretable against the same benchmark context.

6.17 Threats to Validity Built Into the Methodology

Several threats to validity are acknowledged directly in the methodology rather than being postponed to the results chapter alone.

First, some cells operate on modest sample sizes. The methodology therefore treats counts and coverage as part of the result, not as footnotes.

Second, validation support is partial. Pass@1 style results are tied to the supported subset of workflows, and that support boundary is disclosed rather than hidden.

Third, synthetic safety cases may be cleaner than reality. They are therefore disclosed explicitly and used for the parts of the claim that require ground truth rather than for general repair claims.

Fourth, public benchmarks may be contaminated by model pretraining. PatchOps addresses this not by pretending contamination is absent, but by scoping its strongest claims toward safety, abstention, and decision quality instead of headline repair leadership.

6.18 Methodological Honesty Rules

Two practical rules govern the write-up of the later results.

First, when a figure or summary table disagrees with a per-phase findings document, the per-phase `findings.md` takes precedence. This avoids circular drift from re-summarization.

Second, metrics must be read with their relevant context. For example, patch-apply rate must be interpreted alongside coverage, because a selective system can improve apparent quality partly by refusing hard cases. The project records these caveats as part of the methodology itself rather than treating them as optional discussion.

6.19 Chapter Conclusion

This chapter described the evaluation methodology used to study PatchOps across diagnosis, localization, repair, safety, abstention, and cost. The methodology combines a public CI repair benchmark, a labeled safety benchmark, a bounded local validation harness, a modular ablation ladder, and a project-wide pre-registration discipline. It also treats localization, classification, selective prediction, the learned decision gate, and agentic navigation as explicitly measured subproblems rather than informal side effects of one large repair score.

With the datasets, baselines, metrics, and honesty rules now fixed, the next chapter can present the empirical results themselves. Those results will be organized around the thesis’s four headline findings, using the canonical per-phase findings files as the source of truth.

CHAPTER 7

Results and Evaluation

Preface

This chapter presents the empirical results of PatchOps and organizes them around the thesis’s four headline findings: diagnosis quality, the localization bottleneck, the limits of CI-green as a correctness signal, and the need to engineer abstention explicitly.

7.1 Introduction

This chapter presents the empirical results of PatchOps. The results are organized around four headline findings rather than around a chronological list of phases. This structure matters because the project includes both positive gains and honest negative results. The goal of the chapter is therefore not to maximize apparent success, but to show what the experiments establish about diagnosis, localization, repair, safety, abstention, and patch honesty.

Every quantitative claim in this chapter is grounded in the canonical per-phase findings.md files. When summary documents and phase findings differ, the phase findings govern the wording used here. This is particularly important for the localization studies and for the agentic-abstention result, where the correct claim depends on reconciled runs rather than on an early single-run summary.

7.2 Diagnosis Quality and Early Benchmark Behavior

Before PatchOps can repair or abstain intelligently, it has to classify failures reasonably well. The scaled diagnosis ablation ladder provides the clearest diagnosis snapshot on a stratified CI-Repair-Bench subset ($n = 30$) [2]. In that study, category accuracy was 0.83 for the cleaned-log condition and rose to 0.87 when the workflow YAML was added. Importantly, this gain came from workflow context rather than from deeper repository context. The full repository-aware configuration did not improve category accuracy beyond the workflow-aware setting on that slice.

This result is modest rather than dramatic, but it matters. It shows that the diagnosis layer is reasonably competent and that workflow context is useful, while also suggesting that the primary bottleneck of PatchOps is not raw failure categorization. This interpretation is consistent with the overall project trajectory: category accuracy becomes usable relatively early, while later phases reveal that localization and semantic repair remain

substantially harder.

The same diagnosis ablation also exposes category asymmetry. PatchOps is strong on code linting, formatting, dependency issues, runtime errors, package installation, environment errors, assertion errors, documentation/docstring failures, and type checking on that slice. It is weaker on test failures and especially weak on configuration errors. This category split is important because it anticipates the later repair results: mechanical or localized failures are often understandable, while configuration-heavy and semantically ambiguous failures are much harder to route cleanly.

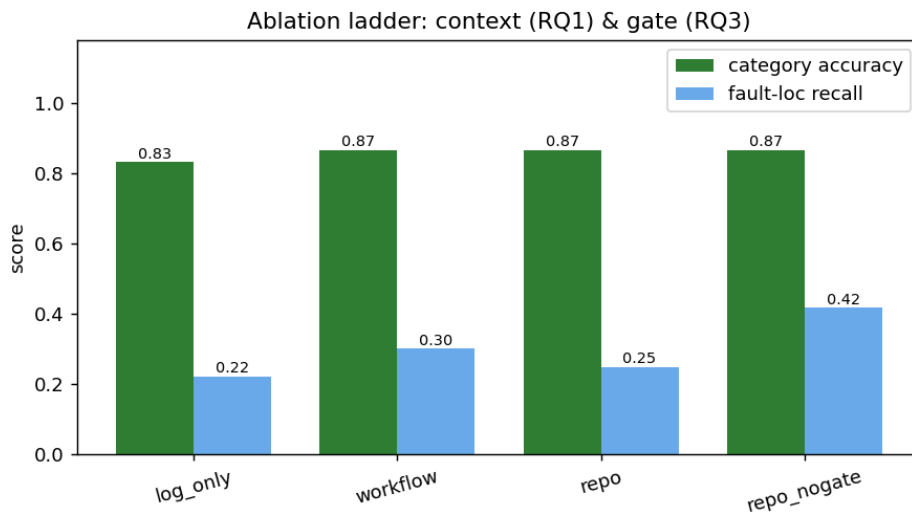


Figure 7.1: Diagnosis ablation ladder showing the effect of context and gating on diagnosis and localization.

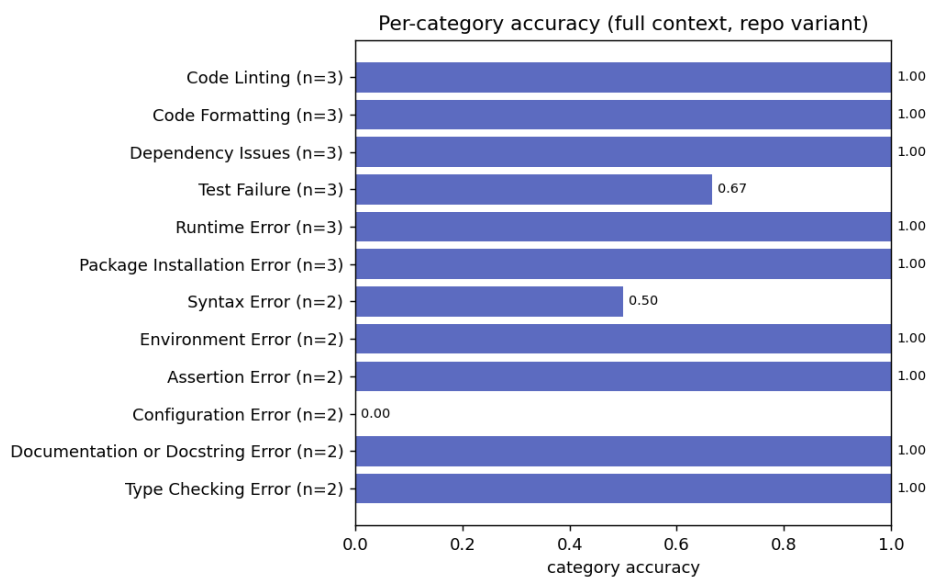


Figure 7.2: Per-category diagnosis behavior on the scaled benchmark slice.

7.3 Fault Localization Remains Hard Across Tested Methods

The strongest multi-study result in the thesis is that fault localization remains the central bottleneck and that no tested method clears the target bar. This finding emerges cumulatively across the log-only, whole-repository, semantic-embedding, and bounded-agentic localization studies.

7.3.1 Log-Only Localization Strategies

The first direct localization study compared four strategies on a stratified CI-Repair-Bench subset ($n = 50$): log-path ranking, category reranking, one-shot LLM prediction, and a hybrid strategy. The best result came from the corrected LLM-first hybrid, which reached Recall@3 of 0.32, Recall@1 of 0.27, and Hit@5 of 0.50. The LLM was the strongest single signal, while the deterministic methods remained cheaper but less complete.

This was a real improvement over the earlier localization red flag around 0.29, but it was also an honest warning. Even the best strategy was still far below the advisor’s target bar of 0.45. The phase therefore established two things simultaneously: localization can be improved by better ranking strategy, but the task is still genuinely difficult.

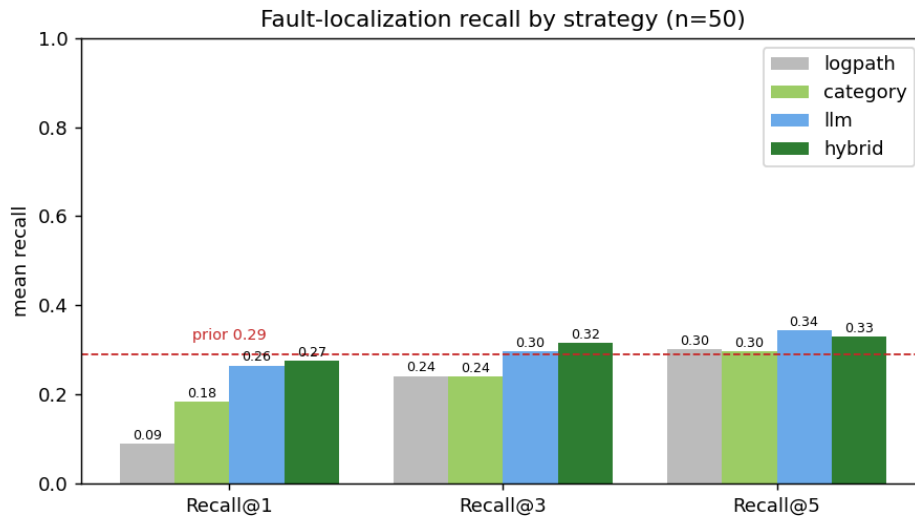


Figure 7.3: Log-only localization strategies. The corrected LLM-first hybrid is strongest but remains below the target bar.

7.3.2 Whole-Repository Retrieval

We next tested whether whole-repository retrieval could break that ceiling. In the deterministic full-repository study ($n = 30$, coverage 1.00), the fused `full_repo` strategy became the best zero-cost retrieval method, reaching Recall@3 of 0.25, Hit@3 of 0.37, and Hit@5 of 0.40. This improved on the best deterministic log-only baseline, but it still did not approach the 0.45 target. A particularly useful sub-finding was that

pure BM25 over the whole repository was actually harmful on its own, collapsing to Recall@3 of 0.06. The retrieval signal only became useful when fused with the precise log-path information.

The same phase also measured a paid LLM lever on top of the retrieval stack. The LLM hybrid reproduced the Recall@3 0.32 result from the earlier log-only study, but an LLM rerank over the BM25 pool did not help whole-repo retrieval. In fact, it reduced the `full_repo` result from Recall@3 0.25 to 0.21. This is one of the most instructive negative results in the project: adding an LLM on top of lexical retrieval did not solve the localization ceiling.

7.3.3 Semantic Embedding Retrieval

We then tested the remaining obvious retrieval lever: semantic embeddings. The result was another honest negative. On the same $n = 30$ draw, `full_repo_embed` reached Recall@3 0.21, which did not beat the deterministic BM25 fusion at 0.25. It traded a little top-3 precision for slightly better depth at rank 5, but it still failed to cross the target bar.

By the end of this retrieval comparison, PatchOps had measured three distinct retrieval families on this task:

1. lexical whole-repository retrieval, peaking at Recall@3 0.25,
2. one-shot LLM reranking or hybridization, peaking around Recall@3 0.27–0.32, and
3. semantic embedding reranking, at Recall@3 0.21.

All three remained below 0.45. This is the point at which the thesis can defend a scoped interpretation: localization in this setting is not merely solved by swapping retrieval methods. Across the methods tested here, simple retrieval is insufficient, and the residual difficulty appears to involve repository reasoning.

7.3.4 Bounded Agentic Exploration

The final major localization lever was bounded agentic exploration over a repository clone. This is where the reconciliation note matters most. The canonical result for the gpt-4o-mini agent on the $n = 30$ seed-7 slice is Recall@3 0.24, with average latency 19.9 seconds per instance and a high forced-fallback rate. The agent therefore slightly edges the deterministic full-repository baseline but does not beat the LLM hybrid and remains far below 0.45.

This result is powerful precisely because it is the fourth negative. After lexical retrieval, LLM one-shot reranking, semantic embeddings, and now agentic exploration,

localization still does not cross the bar. The task appears bottlenecked by repository reasoning difficulty in this evaluated setting rather than by the absence of one more retrieval heuristic.

There is one important nuance. The agent does occasionally show the *possibility* of developer-style exploration helping where ranking fails. On the few instances where it actually commits to a `finish` action rather than timing out into fallback, its committed subset looks stronger than the deterministic baseline, and at least one clean win exists where repository exploration found a relevant fix file the retrieval baseline missed. However, these wins are too sparse and unstable to overturn the global conclusion.

7.3.5 Localization Conclusion

Taken together, these four localization studies support the first headline finding: *across the four methods tested, simple retrieval is insufficient in this setting, and the residual difficulty appears to be repository reasoning*. The best measured values across these families remain clustered well below the target threshold. This does not mean retrieval is useless, and it is not a universal law about all possible localizers. It means that, for PatchOps and this benchmark setting, retrieval is necessary but not sufficient, and the downstream system must remain honest when localization is weak.

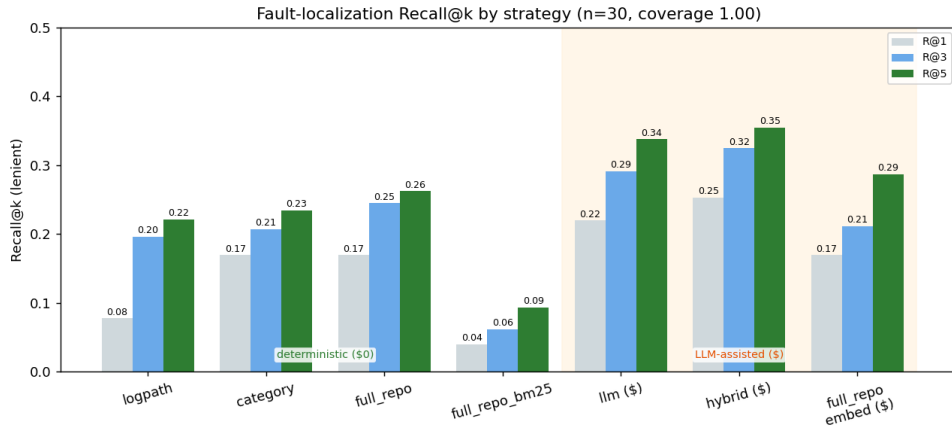


Figure 7.4: Localization strategies across Phases 6, 8, and 10. None clears the 0.45 Recall@3 target.

Comparability note. The best hybrid result is measured on a 50-instance slice, whereas the whole-repository, embedding, and agentic rows are measured on a 30-instance slice. The fine-grained ordering between these families should therefore be read as indicative, while the robust conclusion is the shared plateau below the 0.45 Recall@3 target.

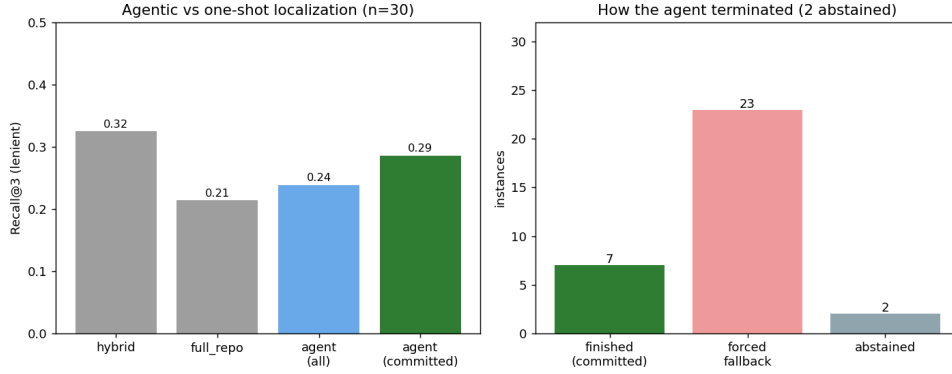


Figure 7.5: Agentic localization result and termination behavior. Agentic exploration is useful in isolated cases but does not clear the localization plateau.

Table 7.1: Headline localization results for the localization bottleneck study.

Method family	Sample	Recall@3	Interpretation
LLM-first hybrid	$n = 50$	0.32	Best overall measured localizer.
Full-repository fused BM25	$n = 30$	0.25	Best zero-cost full-repo retrieval.
Embedding rerank	$n = 30$	0.21	Does not beat lexical retrieval.
Bounded agentic exploration	$n = 30$	0.23–0.24	Does not beat the best hybrid.

7.4 Better Localization Is Necessary but Not Sufficient for Repair

The second headline finding comes from the localization-to-repair linkage experiment. Here the methodology asked a decisive question: if localization improves and patches become cleaner, does CI Pass@1 rise with it?

On the reproducible subset ($n = 24$) of the scaled repair re-run, the ungated `raw_11m` baseline reached Pass@1 0.12 with apply-rate 0.43. The gated `repo` variant, which used oracle changed-files context from the earlier setup, had coverage 0.29, apply-rate 0.29 among attempted cases, and Pass@1 0.00. The key result was the `repo_full` variant: once real clone-based full-repository retrieval was used, apply-rate among attempted cases rose to 1.00, yet Pass@1 stayed 0.00.

These two Pass@1 values are not directly comparable across variants: `raw_11m` is the ungated baseline at 0.96 coverage, whereas `repo_full` is the gated path at 0.29 coverage. The finding rests on the apply-rate-to-Pass@1 decoupling within the gated path, not on a direct cross-variant Pass@1 comparison.

This is a central thesis result because it isolates a specific bottleneck transition. Earlier, one could have suspected that repair was failing mainly because the generated diffs did not even apply cleanly. The linkage study shows that once the apply-rate bottleneck is removed, the system still fails to make CI green on the gated path. The remaining problem is semantic correctness, not simply file selection or diff validity.

In other words, localization improvement is necessary because it prevents invalid or off-tree patches, but it is not sufficient to produce correct repair. This is exactly why the thesis emphasizes honest abstention and safety controls. When the system reaches the semantic frontier, caution becomes more valuable than overconfident automation.

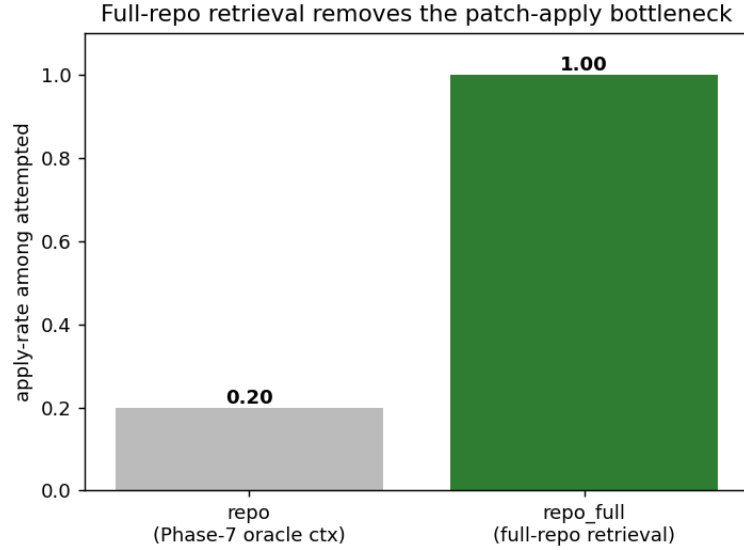


Figure 7.6: Localization-to-repair linkage: full-repository retrieval removes the apply-rate bottleneck among attempted patches, but repair success remains blocked by semantic correctness.

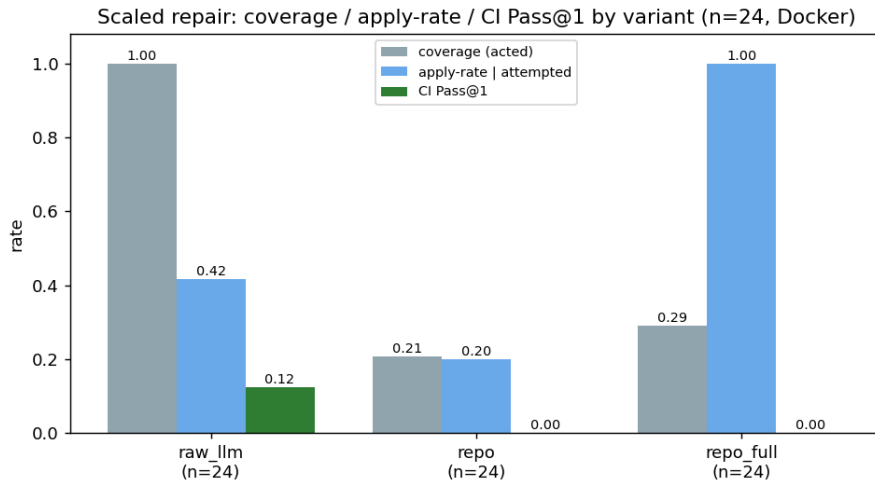


Figure 7.7: Docker repair evaluation on the reproducible subset. Ungated repair obtains some CI-green outcomes, while the gated policy remains conservative.

Table 7.2: Localization-to-repair linkage on the reproducible Docker subset.

Variant	Coverage	Apply-rate among attempted	CI Pass@1
raw_llm	0.96	0.43	0.12
repo	0.29	0.29	0.00
repo_full	0.29	1.00	0.00

7.4.1 Supporting Result: Learned Classification as a Reproducibility and Cost Result

The learned failure classifier adds a narrower but still important result. On a held-out test split of $n = 118$, the learned model reached exact accuracy 0.74, lenient accuracy 0.84, and macro-F1 0.58. The rule baseline, by contrast, achieved exact accuracy 0.37 and lenient accuracy 0.47. The learned model therefore nearly doubled the deterministic rule baseline and landed inside the LLM classifier’s observed lenient accuracy band.

This result should be interpreted carefully. The classifier is not a repair-success breakthrough. Its value is that it makes one part of the pipeline cheaper, more reproducible, and locally trainable while keeping accuracy competitive with the LLM path on the categories that dominate the dataset. This is why it belongs in the results chapter but not as the main headline. It strengthens the system scientifically without pretending to solve the core repair problem.

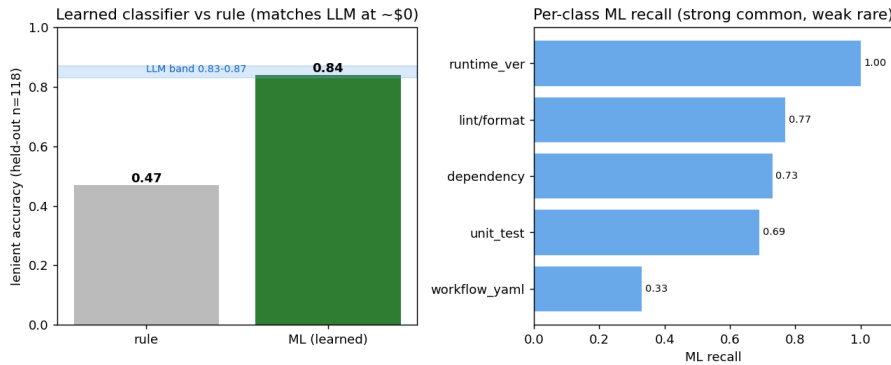


Figure 7.8: Learned classifier compared with the rule baseline. The classifier is a reproducibility and cost contribution, not a repair-rate claim.

7.5 CI-Green Is Not Enough

The third headline finding is that a green or plausible-looking result is not a sufficient indicator of correctness. PatchOps investigates this in two complementary ways: reward-hacking detection and patch plausibility analysis.

7.5.1 Reward Hacking

The reward-hacking audit asked whether any of the 66 produced patches achieved apparent progress by cheating rather than by fixing the underlying issue. The result was small but important: the observed cheat-rate was non-zero, with one detected reward-hacking patch among the 66 produced patches. That patch deleted a test and removed an assertion. The detector caught it, and the wired gate would block it.

Equally important, none of the three CI-green patches in the run were hacky. This lets the thesis state the result honestly: reward hacking is not the dominant behavior of this model on this workload, but it is real enough to require explicit defenses. This is exactly the type of low-frequency, high-consequence behavior a safety-first CI agent should guard against.

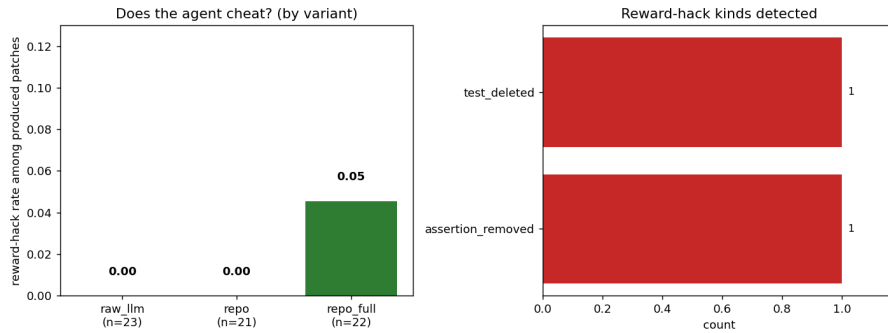


Figure 7.9: Reward-hacking audit. The observed cheat rate is low but non-zero, justifying an explicit detector and gate.

7.5.2 Patch Plausibility

The plausibility study asked a different but related question: even when a patch looks syntactically reasonable and passes the safety gate, is it editing a file that the developer’s actual fix touched? On the 66 produced patches in the validation set, the base on-target rate was only 0.14. In other words, most generated patches were not even editing a gold-overlapping file.

The static plausibility score performed well as a discriminator, reaching AUC 0.77 overall, with evidence locality alone already at 0.75. When used as a gate, it materially improved auto-PR precision: from 0.14 at full coverage to 0.27 at 50% coverage, 0.35 at 25% coverage, and a peak around 0.50 at very low coverage.

This is exactly the patch-level counterpart of the selective-prediction story from the abstention studies. At the diagnosis level, calibrated confidence tells PatchOps when to act. At the patch level, plausibility tells PatchOps when a patch should be previewed instead of being treated as an automatic action candidate.

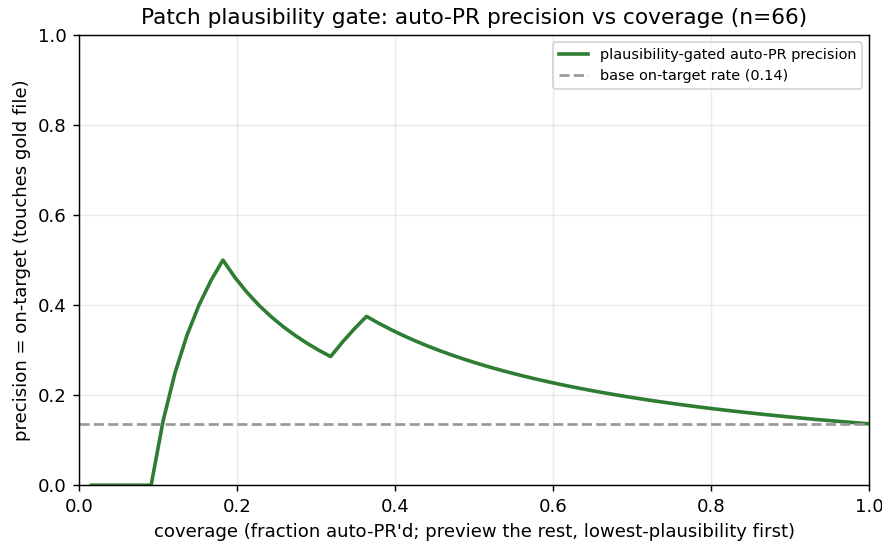


Figure 7.10: Plausibility gate. Precision improves as automatic action is restricted to more plausible patches.

Table 7.3: Patch honesty and plausibility results.

Result	Value	Interpretation
Reward-hack patches	1/66	Low-frequency but real cheating behavior.
CI-green patches that were hacky	0/3	No observed green patch was gamed in this run.
Base on-target rate	0.14	Most produced patches do not edit a gold-overlapping file.
Combined plausibility AUC	0.77	Static signals predict on-target behavior.
Peak gated precision	≈ 0.50	Higher precision is possible at low coverage.

7.6 Abstention Must Be Engineered

The fourth headline finding emerges most clearly in the selective prediction and safety results. The thesis claim is not simply that PatchOps abstains. The claim is that *good abstention must be engineered through external controls rather than left to the free-form model alone*.

7.6.1 Selective Prediction and Calibration

The selective-prediction study turns abstention into a rigorous operating characteristic. On the held-out classification split ($n = 118$), the learned classifier’s confidence produced a strong risk-coverage curve: base accuracy 0.84, AURC 0.051, and selective accuracy rising to 0.97 at 50% coverage and 1.00 at 25% coverage. By contrast, the current heuristic engine confidence was a poor abstention signal, with base accuracy 0.74, AURC

0.261, and erratic selective behavior.

This is a major thesis-quality result. It shows that PatchOps can abstain intelligently when the right confidence signal is used, and that the current hand-rolled confidence should not be trusted as the main gating input. The same study also shows that confidence calibration matters. The learned classifier's raw expected calibration error of 0.224 was cut to 0.119 by isotonic calibration, making the emitted confidence easier to use as a stable decision threshold.

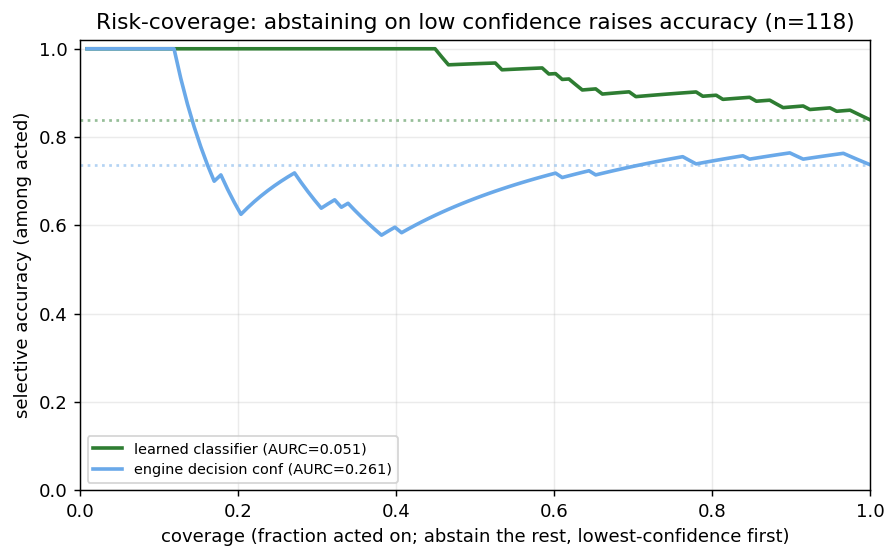


Figure 7.11: Risk-coverage behavior. Learned classifier confidence is a much stronger abstention signal than hand-rolled engine confidence.

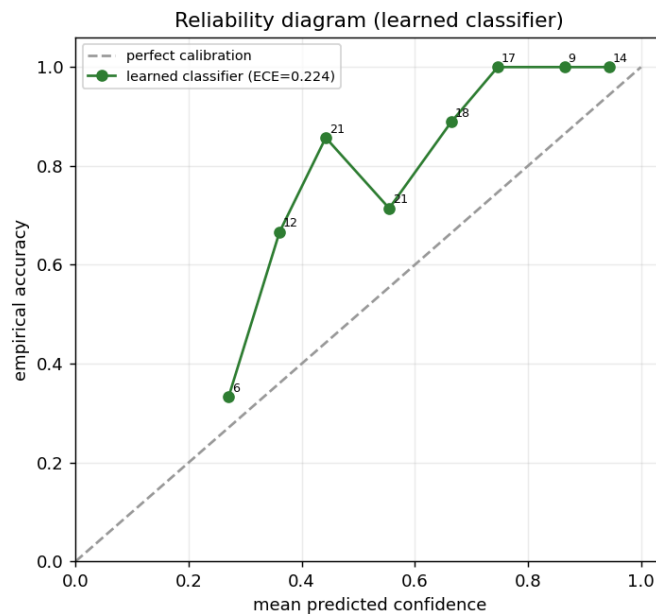


Figure 7.12: Reliability diagram showing the value of isotonic calibration.

7.6.2 The Learned Calibrated Decision Gate

The learned decision-gate study turns the earlier calibration lesson into an implemented decision policy. Earlier PatchOps versions used hand-set constants, including a low confidence floor and a high-confidence patch threshold, to decide when a low-risk case should act or hold. The learned gate replaces the low-risk act-versus-hold threshold with a learned, isotonic-calibrated decision gate over pre-generation signals: calibrated classifier confidence, category risk, flaky status, semantic ambiguity, log-named-file count, and evidence length.

On the expanded Safety Set ($n = 76$), the learned gate reaches act/hold accuracy 0.91 with a Wilson interval of $[0.82, 0.95]$ under leave-one-out evaluation and 0.92 under five-fold evaluation, compared with 0.855 for the hand-tuned threshold policy. It is also well calibrated, with $P(\text{act})$ ECE approximately 0.07. The honest interpretation is matches-or-beats rather than a statistically separated win, because the confidence intervals overlap at this sample size. The important engineering result is that the decision is no longer explained by hand-set thresholds alone.

The most concrete gain appears on the low-risk lane where the engine actually uses the learned gate. Among the 15 cases routed to that lane, all 15 are true-patch cases. The learned gate acts on 15/15, while the hand-tuned thresholds act on only 11/15, recovering four confident patches that the older policy held back, with 0 over-act on that lane. This is a decision-quality result, not a Pass@1 result: it improves the act-versus-hold policy without claiming that semantic repair success is solved.

This result should also be interpreted with its small trade-offs visible. Under leave-one-out evaluation, the learned gate incurs a marginal abstention and over-act cost relative to the hand-tuned policy (0.95 vs. 0.96; 3/56 vs. 2/56), but it gains higher overall act/hold accuracy and recovers low-risk actions that the earlier policy held back.

The same refinement also ships the earlier isotonic calibrator inside the classifier artifact. The emitted classifier confidence ECE falls from 0.18 to 0.06 in the held-out check, making calibrated confidence part of the engine rather than only an offline analysis. A related embedding-versus-TF-IDF classifier ablation is an honest null: neural embeddings reach lenient accuracy 0.66, below the shipped TF-IDF classifier at 0.83. This reinforces the earlier interpretation that classification is not the bottleneck and that simple lexical signals remain strong for this stage.

The two calibration figures in this section refer to different evaluations: the earlier selective-prediction benchmark reports isotonic calibration reducing ECE from 0.224 to 0.119, while the learned decision-gate study reports the shipped classifier artifact on the later held-out check, where ECE falls from 0.18 to 0.06. They are therefore consistent calibration results, not competing measurements of the same run.

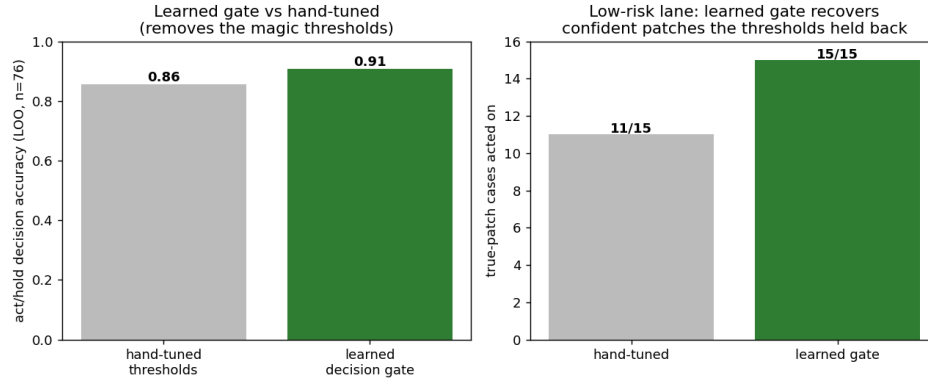


Figure 7.13: Learned decision gate. The calibrated policy matches-or-beats the hand-tuned threshold policy and recovers held-back low-risk actions without over-acting on that lane.

Table 7.4: Learned decision gate summary.

Policy	Act/hold acc.	Abstention acc.	Over-act	$P(\text{act})$ ECE	AURC
Hand-tuned thresholds	0.855	0.96	2/56	—	—
Learned gate, LOO	0.908 [0.82, 0.95]	0.95	3/56	0.072	0.106
Learned gate, 5-fold	0.921 [0.84, 0.96]	0.96	2/56	0.066	0.089

7.6.3 Safety Set Evaluation

The earlier safety-set study already demonstrated the operational value of abstention-oriented behavior. On the labeled $n = 20$ safety benchmark, PatchOps achieved abstention accuracy 0.92, flaky F1 0.86, and 0/20 unsafe auto-fixes. It also correctly abstained on all escalate and rerun cases, with one over-eager preview versus patch mismatch that was later addressed by the semantic ambiguity backstop.

Methodologically, the most important part of this result is not that the benchmark is small, but that the decision classes are explicit. The system is being evaluated on whether it chooses the right kind of action, not only on whether it emits diffs. The Safety Set is later expanded to $n = 76$ for the learned decision-gate study, but the original numbers remain the original 20-case result and should be read with that denominator.

7.6.4 Self-Abstention Nuance

The agentic localization phase supplies the most important cautionary nuance in the whole thesis. The gpt-4o-mini agent was explicitly given an `abstain` action, yet across two runs on the same slice it abstained only rarely and run-dependently: 0/30 in one run and 2/30 in the canonical reconciled run. When those abstentions did occur, they were justified. However, the correct conclusion is not that the model abstains well. The correct conclusion is that self-abstention is too rare and unstable to rely on.

This is why the wording matters. The robust thesis claim is that the agent abstains *rarely and run-dependently*, not never. That instability is itself a finding. It shows that

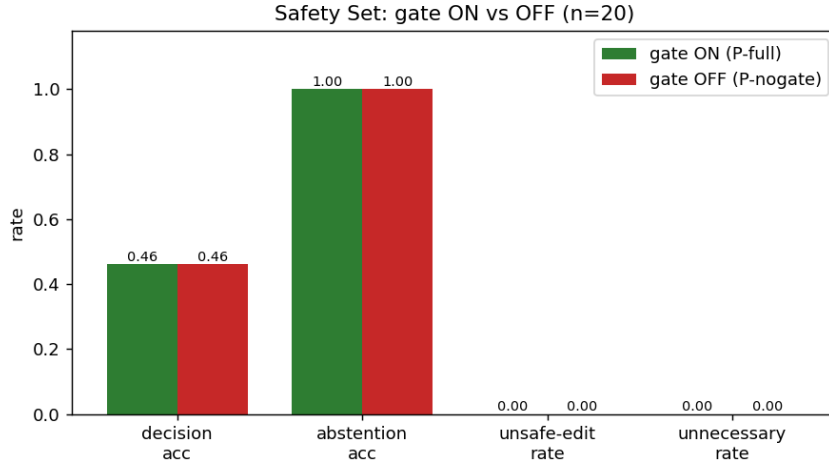


Figure 7.14: Safety Set behavior. The original 20-case result measures decision accuracy, abstention, and unsafe auto-fix behavior.

if PatchOps is to abstain reliably, the abstention logic has to be engineered into the decision stack through calibrated confidence and external safety controls. The learned decision-gate result is the capstone of this argument: the act-versus-hold decision is explicitly learned and calibrated rather than left to self-abstention or hand-set thresholds alone.

7.7 Safety and Practicality as System Outcomes

The combined results also say something broader about the system’s practical operating profile. On the safety benchmarks, PatchOps is conservative and generally asymmetric toward caution. On the repair benchmarks, the ungated baseline can produce more aggressive behavior and occasionally more green outcomes, but at the cost of weak guarantees and lower interpretability. The gated system is more trustworthy, but this trustworthiness is paid for with lower coverage.

This trade-off is not a flaw in the thesis. It is part of the thesis claim. PatchOps is not trying to maximize raw patch volume. It is trying to define a repair agent that behaves sensibly when confidence and correctness signals are weak.

The cost numbers also support this practical framing. The scaled diagnosis ablation remained in the low-dollar range, with approximately \$1.47 total spend for 120 diagnoses using the cheaper model configuration. The later whole-repository retrieval and learned-classifier studies together cost approximately \$0.57. The agentic localization study remained cheap in absolute dollar terms but expensive in latency, averaging about 20 seconds per instance and delivering no Recall@3 gain over the best one-shot hybrid localizer. This makes it clear that agentic exploration is not currently the default practical choice.

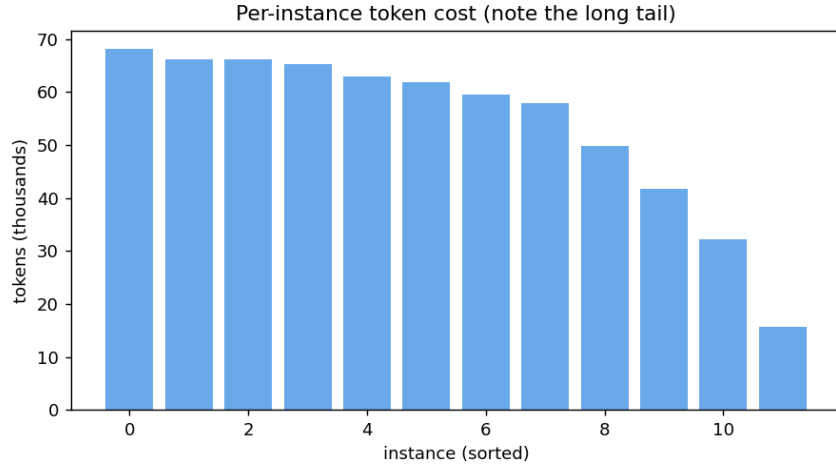


Figure 7.15: Token-cost profile for the scaled evaluation configuration.

7.8 Result Synthesis

The results chapter supports the thesis spine in a coherent way.

First, diagnosis quality becomes serviceable relatively early, especially when workflow context is added, but it does not remove the deeper localization and repair bottlenecks.

Second, localization remains the defining challenge, and repeated attempts to solve it through new retrieval mechanisms and bounded agentic exploration plateau below the desired threshold. This is the strongest evidence that, in the evaluated PatchOps setting, the unresolved part of CI repair appears to involve repository reasoning rather than simple matching alone.

Third, once localization is improved enough to make patches apply cleanly, the next wall is semantic correctness. That transition is one of the clearest and most thesis-worthy findings in the project.

Fourth, confidence and safety controls matter because the system cannot rely on the base agent to abstain reliably, and because CI-green or plausible-looking patches are not trustworthy enough on their own. The learned decision gate strengthens this point by replacing the old low-risk confidence threshold with a learned calibrated gate that matches-or-beats the threshold baseline while recovering held-back low-risk actions.

7.9 Chapter Conclusion

This chapter presented the empirical backbone of the PatchOps thesis. The results show that workflow context improves diagnosis modestly, that localization remains the central bottleneck across four different method families, that fixing localization is necessary but not sufficient for repair success, that a learned classifier can deliver strong reproducibility and cost benefits, that abstention becomes useful only when engineered through calibrated external controls, that the learned decision-gate study turns the act-versus-hold decision

into a learned calibrated gate, and that CI-green is too weak a success criterion without reward-hack and plausibility defenses.

Taken together, these results justify the central thesis claim: PatchOps is best understood not as a system that “solves” CI repair, but as a reproducible, safety-gated, abstention-aware repair agent that remains honest about where repository reasoning and semantic repair are still hard. The next chapter synthesizes these results into the broader discussion and implications of the work.

CHAPTER 8

Discussion

Preface

This chapter interprets the empirical findings, explains why the negative results matter, and positions PatchOps as a conditional repair system rather than a raw patch-generation tool.

8.1 Introduction

The previous chapter reported the empirical results of PatchOps. This chapter discusses what those results mean for the design of AI agents that operate inside CI pipelines. The central point is that PatchOps should not be read as an attempt to maximize patch volume at any cost. It is better understood as a system that makes repair conditional: act when the failure signal, repository evidence, and safety checks support action; otherwise abstain, preview, rerun, or escalate.

This distinction is important because CI repair is an unusually tempting domain for overclaiming. A failed CI run provides an apparently crisp objective, and a green rerun appears to offer a simple success signal. The experiments in this thesis show that the real problem is less tidy. The failure log may identify a symptom rather than the file that should be changed. A patch may apply cleanly but fail semantically. A green CI result may still be produced by an off-target or dishonest edit. An agent may be given the option to abstain and still rarely choose it. These observations are why the safety-gated and abstention-aware framing is not a decorative addition to PatchOps; it is the core design response to the empirical behavior of the task.

8.2 Why the Negative Results Matter

Several of the most important results in PatchOps are negative results. Full-repository lexical retrieval did not break the localization ceiling. LLM reranking did not fix whole-repository retrieval. Semantic embeddings did not outperform the fused lexical baseline. Bounded agentic exploration did not reliably beat cheaper localizers. Full-repository retrieval made generated patches apply, but did not make the gated system achieve CI Pass@1 on the measured repair subset.

These results could be read superficially as a sequence of failed improvements. In a thesis, however, their value is different. They narrow the problem. Before these

experiments, low repair success could have been explained by weak context, missing files, invalid diffs, poor classification, or lack of repository search. The later phases remove several of those explanations. Classification becomes competitive with an LLM through a local learned model. Full-repository retrieval removes the apply-rate bottleneck among attempted gated patches. Embeddings and agentic navigation test the obvious next localization levers. The remaining wall is therefore more specific: semantic repository reasoning and repair correctness.

This sharpening is a real research contribution. A system that only reports end-to-end repair success can hide where the failure occurs. PatchOps decomposes the pipeline into diagnosis, localization, patch production, patch application, validation, safety, and abstention. That decomposition makes it possible to say not only that CI repair is hard, but where and why it remains hard in this implementation.

8.3 PatchOps as a Conditional Repair System

The most important design implication is that CI repair agents should be conditional systems rather than patch-emission systems. A patch-emission system treats every failure as an opportunity to generate a diff. A conditional repair system treats patching as only one possible decision among several. PatchOps therefore distinguishes between `patch`, `preview`, `rerun`, and `escalate`.

This decision vocabulary changes the meaning of success. A correct rerun decision for a flaky failure is not a failure to repair; it is the right action. Escalating a secret, permission, deployment, or ambiguous failure is not a weakness if automatic editing would be unsafe. Previewing a low-plausibility patch is not indecision if the available evidence does not justify opening an automatic PR. In this framing, abstention is not the absence of intelligence. It is one of the system’s safety behaviors.

The results support this design. The labeled safety set shows that explicit decision classes can be evaluated. The selective-prediction experiment shows that abstention can be converted into a risk-coverage trade-off instead of a vague rule. The patch-plausibility experiment shows that even after a patch exists, the system can separate automatic action from preview. The reward-hacking detector shows that a patch can be rejected because it is dishonest, not merely because it fails to apply.

8.4 Why Abstention Must Be Engineered

One of the clearest lessons from the agentic localization study is that abstention cannot simply be delegated to the base model. The bounded repository-exploration agent had an explicit abstain action, but it used it rarely and run-dependently. When it did abstain, the abstentions were justified, but the frequency was too unstable to serve as the main safety mechanism.

This result explains the need for PatchOps’ external decision stack. The learned classifier confidence provides a stronger diagnosis-level abstention signal than the hand-written engine confidence. Phase 15 then turns that calibrated signal into a learned act-versus-hold gate for the low-risk lane, replacing the older hand-set threshold decision. Reward-hacking detection adds a hard veto for dishonest patches. Patch plausibility provides a patch-level signal for whether a diff should be auto-PR’d or merely previewed. Together, these components make abstention a property of the system architecture rather than a hopeful behavior of the LLM.

This distinction also matters for accountability. If an LLM freely decides when to decline, the system has only a weak explanation for why it acted on one failure and not another. If abstention is tied to calibrated confidence, a learned act-versus-hold model, safety predicates, and patch-locality signals, the system can produce a traceable reason: low diagnosis confidence, high-risk edit target, reward-hacking pattern, or low plausibility. That traceability is essential for a CI agent that may open PRs in a real repository.

8.5 CI-Green as a Partial Oracle

CI validation is valuable, but it is not a complete correctness oracle. PatchOps uses CI re-execution because it is the strongest readily available feedback signal in this domain. If a generated fix still leaves CI red, the system has direct evidence that the repair failed. However, the reverse does not always hold. A green run may mean the underlying problem was fixed, but it may also mean the check was weakened, the failing test was removed, or the patch happened to satisfy the current test suite while missing the semantic intent.

This is why the thesis connects CI repair to the broader APR overfitting literature. The risk is not only that an agent fails to fix a bug. The more subtle risk is that it produces a plausible patch that optimizes the observable validation signal without respecting the intended behavior. PatchOps’ reward-hack and plausibility gates are responses to that risk. They do not prove semantic correctness, but they make the system less willing to treat every green or syntactically valid patch as acceptable.

The phrase “CI-validated repair” should therefore be read carefully. It means that selected fixes are checked against CI, not that CI fully certifies correctness. The thesis contribution is the combination of CI validation with safety and abstention controls, not a claim that CI alone is enough.

8.6 The Role of the Learned Classifier

The learned classifier has a deliberately modest role in the thesis. It is not presented as the main technical breakthrough and it is not claimed to improve CI Pass@1. Its value is

practical and methodological. It makes one stage of the pipeline cheaper, reproducible, and trainable from public data while matching the LLM’s observed classification band on the measured split.

This matters because CI agents are often evaluated under conditions that are difficult to reproduce exactly: closed models, drifting APIs, hidden prompts, and paid inference. Moving classification onto a small local model does not remove all of those issues, because the rest of PatchOps still uses LLMs for generation and reasoning. However, it reduces reliance on closed inference for a repeated pipeline stage and provides a stable confidence signal for selective prediction.

The classifier is therefore best understood as infrastructure for trustworthy operation. It supports the abstention story, improves cost behavior, and strengthens reproducibility. It does not distract from the main finding that localization and semantic repair remain the hard parts.

8.7 The Learned Decision Gate as the Abstention Capstone

Phase 15 resolves a weakness that an examiner would naturally notice in the earlier architecture: if the system uses confidence thresholds, where did those thresholds come from? The answer in the final PatchOps artifact is that the low-risk act-versus-hold decision is no longer only a hand-set threshold. It is an optional learned, calibrated decision gate trained on the labeled Safety Set.

This matters because it converts the phrase “abstention must be engineered” into an implemented mechanism. The gate uses signals available before patch generation and produces a calibrated probability of action. On the expanded 76-case Safety Set it matches-or-beats the hand-tuned threshold policy, with overlapping confidence intervals, and on the low-risk lane it recovers confident patch actions that the old thresholds held back. The correct discussion point is not that PatchOps suddenly repairs more CI failures end to end. The point is that the act-versus-hold boundary is now measured, calibrated, and inspectable.

The result also preserves the conservative structure of PatchOps. The learned gate does not bypass flaky reruns, high-risk escalation, reward-hack detection, or patch plausibility. It only decides whether a low-risk case should proceed toward action or remain held for preview. This is the right level of learning for the thesis claim: use learned calibration to remove brittle constants, but keep deterministic safety boundaries in place.

8.8 Architecture: Why a Staged Pipeline Is Preferable Here

PatchOps uses a staged decision pipeline rather than an unconstrained multi-agent architecture. The experiments support that choice. In CI repair, each stage has a different failure mode: diagnosis can misclassify the failure, localization can retrieve the wrong

file, patch generation can overfit, CI validation can be gamed, and the planner can act too aggressively. A staged architecture lets the system attach checks and evidence to each boundary.

The agentic localization phase is especially informative here. Tool use is attractive because it resembles how developers investigate repositories: search, read, follow imports, and decide. In the measured setting, however, the bounded agent was slower than deterministic retrieval and did not provide a global localization gain. That does not mean agentic exploration is useless; the clean wins show that it can help in some cases. It does mean that agentic behavior should be introduced as a bounded component with fallback and external gates, not as an unrestricted replacement for the whole pipeline.

The practical design lesson is that AI CI repair benefits from explicit control surfaces. Deterministic retrieval, learned classification, the learned decision gate, LLM reasoning, safety gates, plausibility checks, and validation runners each contribute different kinds of evidence. The thesis argues for orchestrating these pieces conservatively rather than trusting a single model call to diagnose, localize, patch, validate, and self-regulate.

8.9 Implications for Deployment

In a real engineering team, PatchOps would be most appropriate as a tiered assistant rather than a fully autonomous maintainer. For low-risk, log-localizable failures such as straightforward version bumps or narrowly scoped dependency issues, the system can propose or open a small PR after safety checks. For ambiguous failures, test-semantics failures, permissions issues, deployment failures, and low-plausibility patches, the system should provide a preview or escalation with evidence instead of changing code automatically.

This operating model is less glamorous than a universal auto-fixer, but it is more realistic. CI pipelines are part of the trust boundary of a repository. A bad patch can hide a real regression, weaken tests, or modify workflow behavior in ways that have security and release consequences. The correct default is therefore not “always patch.” The correct default is “patch only when the evidence and gates justify it.”

The results also suggest how teams could tune PatchOps. A team that prioritizes speed could lower abstention thresholds or the learned gate’s operating point and accept more previewed patches. A team that manages critical infrastructure could require higher plausibility and confidence before auto-PR. The important point is that PatchOps exposes this as a measurable coverage-versus-risk trade-off rather than as a hidden prompt preference.

8.10 Relation to Prior CI Repair Work

The discussion also clarifies how PatchOps should be positioned relative to CI-Repair-Bench and related repair systems. PatchOps does not claim to be the first system to perform CI-verified repair, and it does not claim to beat the strongest reported repair baselines on raw Pass@1. Those would be the wrong claims. CI-Repair-Bench already establishes repository-level CI-verified repair as a benchmarked task, and its strongest baseline sets a clear reference point.

PatchOps contributes along a different axis. It asks what an agent should do when repair evidence is weak, when patches are plausible but off-target, when the validation oracle can be gamed, and when the model does not reliably abstain on its own. This safety-and-abstention axis is not a substitute for repair accuracy; it is a complement to it. In practical CI automation, both are needed. A high-repair system that cannot distinguish safe from unsafe action is risky, while a safe system with low coverage is useful only on a narrower subset. PatchOps occupies the second side of that trade-off and makes the trade-off explicit.

8.11 Chapter Conclusion

This chapter interpreted the empirical results of PatchOps as evidence for conditional, safety-gated CI repair. The main discussion points are that negative results help isolate the true bottleneck, abstention must be engineered rather than entrusted to the model alone, Phase 15 turns the key low-risk act-versus-hold boundary into a learned calibrated gate, CI-green is only a partial correctness signal, and the staged architecture is justified by the distinct failure modes of diagnosis, localization, generation, validation, and decision-making.

The next chapter states the limitations and threats to validity of these claims. It is intentionally explicit about scope, sample sizes, benchmark dependence, proxy metrics, and the gap between this thesis prototype and a production CI system.

CHAPTER 9

Threats to Validity and Limitations

Preface

This chapter examines the main threats to validity, including dataset scope, benchmark coverage, synthetic-heavy safety labels, CI validation limits, and the boundaries of the thesis claims.

9.1 Introduction

This chapter describes the main threats to validity and limitations of the PatchOps thesis. The goal is not to weaken the contribution, but to state precisely where the evidence applies. PatchOps is an MTech-scale research prototype evaluated on bounded datasets and controlled harnesses. Its strongest claims concern safety-gated decision-making, abstention, localization bottlenecks, and measured negative results. Its weaker claims concern general repair success, broad production readiness, and semantic correctness beyond the available oracles.

The limitations are organized using four standard categories: construct validity, internal validity, external validity, and conclusion validity. The chapter then summarizes the practical engineering limitations of the current system.

9.2 Construct Validity

Construct validity asks whether the thesis measures the concepts it claims to measure. In PatchOps, this issue is most visible around repair success, localization quality, safety, and patch plausibility.

9.2.1 CI Pass Is Not Full Correctness

The thesis uses CI Pass@1 because it is the natural benchmark signal for CI repair. However, a passing CI run is not equivalent to semantic correctness. A patch may pass the current tests while violating intended behavior, weakening assertions, deleting checks, or changing only the symptom. This limitation is fundamental to CI repair and APR, not only to PatchOps.

PatchOps mitigates this threat by explicitly separating CI validation from semantic correctness. The thesis uses reward-hacking detection and patch-plausibility analysis

to avoid treating every green or syntactically valid patch as trustworthy. Even so, these defenses are partial. They can catch specific dishonest or off-target patterns, but they cannot prove that a patch is functionally equivalent to the developer’s intended fix.

9.2.2 File-Overlap Is a Proxy for On-Target Repair

The plausibility analysis uses overlap with the developer’s changed files as a proxy for whether a generated patch is on target. This is useful because it is objective and scalable, but it is imperfect. A correct patch may edit a different file from the developer fix, especially if there are multiple legitimate repair locations. Conversely, a patch may edit a gold-overlapping file while still being semantically wrong.

For this reason, the thesis describes plausibility as an on-target signal rather than as a correctness oracle. The result supports a safer auto-PR decision, not a proof of repair correctness.

9.2.3 Localization Metrics Capture Ranking, Not Understanding

Recall@k and Hit@k measure whether the true fix file appears in a ranked candidate list. These metrics are appropriate for comparing localizers, but they do not fully capture repository understanding. A localizer may rank the right file because of a lexical coincidence, or it may understand the failure but rank a nearby file outside the gold set. This affects all localization methods in the thesis.

The mitigation is comparative rather than absolute. PatchOps evaluates lexical, LLM, embedding, and agentic approaches under the same gold-file criterion. The conclusion that these methods plateau below the target bar is therefore meaningful for the measured setup, even if the metric is not a complete measure of reasoning.

9.2.4 Safety Labels Are Operational

Safety in this thesis is defined operationally: forbidden file classes, secret or permission risks, deployment risks, flaky rerun cases, workflow weakening, reward-hacking patterns, and inappropriate automatic patching. This makes the safety benchmark measurable, but it does not cover every possible safety issue in a real repository. For example, a patch could be business-logic unsafe without touching an obviously forbidden file.

The thesis therefore claims measured safety under defined policies, not universal safety. This is still useful: CI agents need explicit policies, and PatchOps demonstrates how such policies can be integrated into the repair decision.

9.3 Internal Validity

Internal validity asks whether the observed results are caused by the intended experimental factors rather than by bugs, leakage, or uncontrolled variation.

9.3.1 Pipeline Bugs and Regression Risk

PatchOps is a multi-stage system. A bug in log extraction, repository cloning, ranking, diff generation, or decision routing could affect multiple reported metrics. The project mitigates this risk through regression tests, acceptance tests, per-phase artifacts, and explicit amendments when bugs are discovered. For example, the localization ordering issue was corrected before the later localization conclusions were frozen.

Nevertheless, implementation bugs remain possible. The thesis should be read as evidence from the current artifact, not as a proof that every pipeline component is bug-free.

9.3.2 Model Non-Determinism

LLM calls are not perfectly reproducible. Even with low-temperature settings and fixed prompts, provider-side model updates, decoding differences, and transient API behavior can change outputs. This risk is visible in the agentic localization phase, where abstention behavior differed across runs.

PatchOps mitigates this risk by logging phase results, using deterministic components where possible, and adding a learned local classifier for the classification stage. The limitation remains for LLM-based localization and patch generation. Future repeated-run studies would strengthen variance estimates.

9.3.3 Local CI Reproduction

The Docker validation harness replays a restricted subset of GitHub Actions behavior. It is not a full GitHub Actions emulator. Differences in hosted runners, caches, service containers, custom actions, secrets, matrix jobs, and operating-system behavior can affect whether a failure reproduces locally.

The thesis handles this by reporting reproducibility separately from repair success. Non-reproducible cases are not silently counted as repaired or failed repairs. However, the repair results apply only to the reproducible subset under the local harness.

9.3.4 Budget and Provider Constraints

Some experiments use cost-bounded model calls. This is appropriate for a practical thesis, but it also affects which methods are explored deeply. The agentic localizer, for example, was evaluated under a bounded step and budget regime with mid-tier and limited stronger-model probes. A larger budget, stronger reasoning model, or more sophisticated tool interface could change the absolute localization numbers.

The thesis therefore frames the agentic result as an honest negative for the measured configuration, not as a universal impossibility result for all future agentic localization.

9.4 External Validity

External validity asks whether the results generalize beyond the evaluated datasets, languages, and repositories.

9.4.1 Python-First Benchmark Scope

The comparable benchmark results are Python-first because CI-Repair-Bench is Python-based. PatchOps also has live demonstration cases outside that exact benchmark scope, but those are treated as demonstrations rather than as benchmark evidence. The results should not be generalized automatically to JavaScript, Java, Go, Rust, mobile builds, or polyglot monorepos.

The main architectural ideas—safety gates, abstention, CI validation, plausibility checks, and reward-hack detection—are language-agnostic in principle. The measured effectiveness of the current implementation is not.

9.4.2 GitHub Actions Scope

PatchOps targets GitHub Actions. Other CI systems such as GitLab CI, Jenkins, CircleCI, Buildkite, and Azure Pipelines have different configuration languages, permission models, caching behavior, log formats, and failure patterns. The thesis findings may transfer conceptually, but the implementation and taxonomy would need adaptation.

9.4.3 Benchmark Representativeness

CI-Repair-Bench provides real failures and is an appropriate benchmark for this project, but no benchmark can cover the full distribution of CI failures in the wild. The public benchmark is also low-risk dominated compared with the kinds of secret, permission, release, and deployment failures where abstention policy matters most. PatchOps addresses this by adding a labeled safety set, but that set is smaller and partly synthetic.

As a result, the repair and localization results have stronger external validity for public Python CI failures than for high-risk enterprise pipelines. The safety results demonstrate policy behavior under defined cases, but larger real-world safety datasets would be needed for stronger generalization.

9.4.4 Synthetic-Heavy Decision-Gate Data

The Phase 15 learned decision gate uses the expanded 76-case Safety Set. This directly improves coverage over the original 20-case safety set, but it also introduces an important validity threat: most of the expanded cases are hand-authored synthetic cases. Such data is useful for testing policy behavior under labeled risk conditions, including adversarial reward-hack temptations, but it may also encode the author’s labeling shortcuts. The

Phase 15 result should therefore be read as an indicative labeled-set result, not as a field measurement from deployed CI pipelines.

9.4.5 Repository Size and Complexity

The experiments include real repositories, but repository complexity varies widely in practice. Very large monorepos, generated code, vendored dependencies, custom build systems, and service-backed integration tests may change both localization and validation behavior. The current retrieval and agentic tools are bounded and may not scale linearly to such settings.

9.5 Conclusion Validity

Conclusion validity asks whether the evidence is strong enough to support the stated conclusions.

9.5.1 Sample Sizes

Several experiments have modest sample sizes. The scaled diagnosis ablation uses $n = 30$, the repair linkage uses $n = 24$ reproducible cases, the original Phase 5 safety result uses $n = 20$, the Phase 15 decision-gate study uses $n = 76$, and the patch audit uses 66 produced patches. The held-out classifier and selective-prediction split is larger at $n = 118$, but it still does not eliminate sampling uncertainty.

The thesis mitigates this by avoiding over-precise claims and by reporting negative results conservatively. For example, it does not claim that agentic localization can never work; it claims that the measured bounded agent does not beat the baselines on the evaluated slice. It does not claim zero universal risk; it reports zero unsafe auto-fixes under the evaluated safety definitions.

For the learned decision gate specifically, the Wilson confidence intervals overlap with the hand-tuned threshold baseline at $n = 76$. The thesis therefore uses the phrase “matches-or-beats” and emphasizes removal of the hand-set thresholds rather than claiming a statistically separated victory.

9.5.2 Multiple Experiments and Narrative Selection

Because PatchOps went through many phases, there is a risk of selecting only the most favorable results. The writing mitigates this by foregrounding negative results and preserving the phase chronology in the source documents. The thesis narrative is built around the measured bottlenecks, not around a cherry-picked success case.

9.5.3 Threshold Choices

Several gates depend on thresholds: classifier confidence, plausibility score, risk level, and decision routing. Thresholds are necessary for a deployable system, but the exact defaults may not be optimal. The thesis treats them as operating points on a coverage-risk curve rather than as universal constants. Phase 15 reduces this threat for the low-risk act-versus-hold decision by replacing the old hand-set threshold with a learned calibrated gate, but the learned gate still has its own operating point and inherits the limitations of the Safety Set used to train and evaluate it.

9.6 Practical System Limitations

PatchOps is not production-ready in the broad sense. It demonstrates an end-to-end architecture and measured research claims, but several engineering limitations remain.

First, the local validation harness supports only a restricted subset of common Python CI workflows. A complete system would need stronger handling for matrix builds, service containers, custom actions, caches, secrets, container jobs, and operating-system differences.

Second, the current repair generator is not a strong semantic repair engine. The experiments show that once patches apply, semantic correctness remains the wall. Future work needs stronger program understanding, differential testing, invariant checking, or developer-in-the-loop review.

Third, the repository navigator is bounded and model-limited. It provides useful evidence, but it is not yet a robust replacement for deterministic retrieval. Better tools, graph or AST structure, and stronger reasoning models may improve it.

Fourth, the safety policy is necessarily incomplete. It captures important CI-specific hazards, but each organization would need to adapt forbidden paths, protected workflows, deployment rules, and approval policies.

9.7 Chapter Conclusion

The main limitations of PatchOps are bounded scope, modest sample sizes, proxy correctness metrics, local CI reproduction limits, and incomplete generalization beyond Python GitHub Actions. These limitations do not invalidate the central contribution, because the thesis is careful about what it claims: PatchOps is a measured safety-gated and abstention-aware CI repair system, not a universal CI repair solver. The next chapter concludes the thesis and outlines future work that follows directly from these limits.

CHAPTER 10

Conclusion and Future Work

Preface

This chapter summarizes the thesis findings, restates the main contributions, and outlines future work needed to move PatchOps toward stronger semantic repair and operational deployment.

10.1 Conclusion

This thesis presented PatchOps, a repository-aware, safety-gated, abstention-aware agent for GitHub Actions CI failures. The project began from a practical goal: diagnose a failed CI run, identify an appropriate repair action, generate a candidate fix when safe, validate it through CI execution, and avoid harmful automation when the evidence is weak. The final contribution is more specific and more defensible than simply “an AI that fixes CI.” PatchOps is an AI repair system that treats *when not to fix* as a first-class research problem.

The thesis deliberately avoids claiming to be the first CI-verified repair system or to outperform the strongest repair baselines on raw Pass@1. Prior CI repair work already establishes repository-level CI-validated repair as a benchmarked task. PatchOps instead contributes a safety-and-abstention view of the problem, supported by a working system and a sequence of measured experiments.

10.2 Summary of Contributions

The first contribution is the PatchOps architecture itself: a staged pipeline for CI diagnosis, repository context retrieval, root-cause reasoning, action planning, patch generation, safety gating, validation, and decision routing. The system distinguishes between patching, previewing, rerunning, and escalating rather than treating every failure as a patch-generation task.

The second contribution is an empirical decomposition of CI repair failure modes. The experiments show that classification becomes reasonably tractable, while fault localization and semantic repair remain difficult. Across lexical retrieval, LLM hybrid ranking, semantic embedding retrieval, and bounded agentic exploration, localization plateaus below the target bar. Full-repository retrieval improves patch applicability, but does not make gated repair achieve CI Pass@1 on the measured repair subset. This

isolates the remaining bottleneck as semantic repository reasoning and repair correctness rather than merely missing files or invalid diffs.

The third contribution is a measured abstention and safety stack. The learned classifier provides a local, reproducible, low-cost diagnosis stage and a useful confidence signal for selective prediction. Calibration makes that confidence more usable, and Phase 15 turns the low-risk act-versus-hold decision into a learned, calibrated gate that replaces the earlier hand-set threshold. The reward-hacking detector asks whether a CI-green outcome is honest, and the patch-plausibility gate asks whether a patch is likely to be on target before it is treated as an automatic PR candidate. Together, these components turn abstention from a prompt instruction into an engineered system behavior.

The fourth contribution is claim discipline. The thesis separates live demonstrations from benchmark evidence, separates CI validation from semantic correctness, and reports low or negative results rather than hiding them. That honesty is part of the scientific value of the work: it makes clear where CI repair agents are useful today and where they remain unreliable.

10.3 Answers to the Research Questions

RQ1 asked whether repository and workflow context improve diagnosis and localization. The results show that workflow context modestly improves diagnosis, while deeper repository context is more important for file candidate discovery than for category classification. However, repository retrieval alone does not solve localization.

RQ2 asked whether PatchOps patches apply and improve CI repair success compared with simpler baselines. The answer is mixed and intentionally conservative. Full-repository retrieval improves patch applicability among attempted gated patches, but gated CI Pass@1 remains low on the measured reproducible subset. The raw ungated baseline can produce some green outcomes, but without the same safety guarantees.

RQ3 asked whether the safety-gated decision policy reduces unsafe or inappropriate repair attempts. The results support this as the strongest thesis claim. PatchOps explicitly abstains, reruns, previews, or escalates when automatic patching is not justified, and the later gates add protection against reward hacking and off-target patches. Phase 15 further strengthens the decision-policy result by showing that the low-risk act-versus-hold boundary can be learned and calibrated: it matches-or-beats the hand-tuned threshold baseline on the expanded Safety Set and recovers held-back confident low-risk actions without over-acting on that lane.

RQ4 asked whether PatchOps can distinguish repairable failures from transient or flaky cases. The labeled safety set shows useful flaky handling and abstention behavior, but the sample remains limited. The conclusion is that PatchOps has a working decision mechanism for rerun-versus-repair, not that flaky classification is fully solved.

RQ5 asked about cost and practicality. The measured pipeline remains low-cost in

the main offline experiments, and the learned classifier reduces paid-model dependence for classification. Agentic exploration, however, adds significant latency without a global localization gain in the measured configuration. This supports the choice of a conservative staged pipeline with selective use of expensive reasoning.

10.4 Future Work

The most important future direction is semantic repair. PatchOps can now make patches apply more cleanly, but applicability is not correctness. Future systems should integrate stronger program analysis, differential testing, invariant checking, and richer execution feedback so that the repair loop can reason about behavior rather than only about CI outcomes and file overlap.

A second direction is stronger verification. The current reward-hack and plausibility gates are useful but partial. Future work should add semantic-equivalence checks, mutation-based validation, test-impact analysis, and stronger patch-correctness oracles. This would make “CI-green” one signal among several rather than the dominant proxy.

A third direction is improved repository navigation. The bounded agentic localizer shows that exploration can find files missed by retrieval in individual cases, but it is not reliable enough yet. Stronger reasoning models, graph-guided tools, AST and import-following support, and forced confidence-based exits may make agentic localization more useful. Future studies should measure these methods against the same localization and latency baselines rather than assuming agentic navigation is automatically better.

A fourth direction is broader reproducibility. The learned classifier is already local and reproducible, but the LLM-dependent stages should be evaluated across stronger commercial models and open-weight models with repeated runs. This would clarify which conclusions are architecture-level and which depend on a particular model tier.

A fifth direction is larger and more realistic safety evaluation. The current safety set is useful, but real enterprise CI failures include secrets, protected deployments, permissions, release workflows, flaky infrastructure, and organization-specific rules. Although the Safety Set has been expanded to support the Phase 15 learned gate, it remains synthetic-heavy. A larger real-derived labeled safety benchmark would strengthen the abstention claim and make the safety policy more transferable.

A sixth direction is CI-system coverage. PatchOps currently targets GitHub Actions and a restricted local validation harness. Extending the approach to richer GitHub Actions features, and eventually to GitLab CI, Jenkins, or other CI systems, would test how much of the architecture transfers beyond the current setting.

10.5 Final Remarks

PatchOps does not solve CI repair, and the thesis does not need it to. The project shows something more useful for the current state of LLM-based software agents: a CI repair agent must know when not to act. The experiments demonstrate that localization remains difficult, semantic repair remains unresolved, CI-green is only a partial oracle, and model self-abstention is not reliable enough by itself. In response, PatchOps builds abstention, a learned calibrated decision gate, safety gates, plausibility checks, and validation into the repair loop.

The final lesson is therefore practical as well as scientific. Future CI agents should not be judged only by how often they generate patches. They should be judged by whether they choose the right action under uncertainty, whether their green results are honest, whether they expose confidence and risk, and whether they decline to act when the evidence does not justify automation. PatchOps is a step toward that kind of repair agent: not an overconfident auto-fixer, but a cautious, evidence-aware collaborator inside the CI workflow.

APPENDIX A

Reproducibility Artifacts

A.1 Purpose

This appendix records the main files that support the PatchOps thesis results. It is intended as a navigation aid for examiners and future maintainers rather than as a replacement for the full project repository. The thesis follows the rule that per-phase `findings.md` files are the canonical source of quantitative claims.

A.2 Canonical Result Sources

Table A.1: Canonical result artifacts used by the thesis.

Artifact	Role in thesis
<code>docs/THESIS_WRITING_GUIDE.md</code>	Thesis spine, claim discipline, chapter plan, and required Phase 15 updates.
<code>docs/PROGRESS_REPORT.md</code>	Master status summary and contribution framing.
<code>docs/results/metrics_plan.md</code>	Table and figure catalog, metric definitions, and current result values.
<code>docs/progress/phase*/findings.md</code>	Canonical quantitative results for each phase.
<code>docs/progress/phase*/conclusion.md</code>	Interpretation and scope of each phase result.
<code>data/eval/*.jsonl</code>	Raw evaluation rows used by the benchmark scripts and figure generator.

A.3 Benchmark and Analysis Scripts

Table A.2: Experiment scripts referenced by the methodology and results chapters.

Script	Purpose
benchmarks/run_eval.py	Diagnosis, decision, and ablation experiments.
benchmarks/run_localization.py	Log-path, hybrid, full-repository, and embedding localization studies.
benchmarks/run_repair_eval.py	Docker-based Pass@1 repair evaluation.
benchmarks/train_classifier.py	Learned failure classifier and calibration artifact.
benchmarks/run_selective.py	Risk-coverage and calibration experiments.
benchmarks/audit_reward_hack.py	Reward-hacking audit over produced patches.
benchmarks/analyze_plausibility.py	Patch plausibility and on-target analysis.
benchmarks/run_agent_localization.py	Bounded ReAct repository-navigation localizer.
benchmarks/train_decision_model.py	Phase 15 learned act-versus-hold decision gate.
benchmarks/make_figures.py	Regenerates the figures copied into Figures/.

A.4 Engine Modules

Table A.3: Core implementation modules.

Module	Responsibility
patchops/engine/decide.py	Main state machine and final action selection.
patchops/engine/log_extractor.py	CI log compression and evidence extraction.
patchops/engine/classifier.py	Rule/LLM/learned failure classification policy.
patchops/engine/ml_classifier.py	Optional trained classifier and shipped calibrator.
patchops/engine/localizer.py	Fault-localization strategies.
patchops/engine/repo_index.py	Full-repository BM25 and embedding reranking.
patchops/engine/agent_localizer.py	Bounded agentic repository exploration.
patchops/engine/safety_gate.py	Deterministic patch safety checks.
patchops/engine/reward_hack.py	Test-tampering and CI-gaming detection.
patchops/engine/patch_plausibility.py	On-target and overfit-risk score.
patchops/engine/decision_gate.py	Phase 15 learned calibrated act-versus-hold gate.
patchops/validation/	Local CI-style re-execution harness.

Bibliography

- [1] LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation. ASE 2025 Industry Showcase; arXiv:2506.03691, 2025.
- [2] CI-Repair-Bench: A Repository-Aware Benchmark for Automated Patch Validation via CI Workflows. arXiv:2604.27148, 2026.
- [3] HireBuild: An Automatic Approach to History-Driven Repair of Build Scripts. Proceedings of the International Conference on Software Engineering, 2018.
- [4] DeepDelta: Learning to Repair Compilation Errors. Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019.
- [5] A Survey of LLM-Based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. arXiv:2506.23749, 2025.
- [6] A Systematic Literature Review on Large Language Models for Automated Program Repair. arXiv:2405.01466, 2024.
- [7] SWE-bench: Can Language Models Resolve Real-World GitHub Issues? Repository-level software engineering benchmark, 2023.
- [8] Agentless: Demystifying LLM-Based Software Engineering Agents. Structured localization and repair pipeline for SWE-bench-style tasks, 2025.
- [9] GHALogs: A Large-Scale Dataset of GitHub Actions Workflow Runs and Logs. MSR 2025 dataset; Zenodo record 10154920, 2025.
- [10] GitBug-Actions: Building Reproducible Benchmarks for GitHub Actions Bugs. Proceedings of the International Conference on Software Engineering, 2024.
- [11] Why Do GitHub Actions Workflows Fail? ACM Transactions on Software Engineering and Methodology, 2025.
- [12] Understanding and Detecting Flaky Builds in GitHub Actions. arXiv:2602.02307, 2026.
- [13] Is the Cure Still Worse Than the Disease? Test Overfitting by Large Language Models in Automated Program Repair. arXiv:2511.16858, 2025.
- [14] ComPass: Contrastive Patch Correctness Assessment. arXiv:2602.07561, 2026.

- [15] Know Your Limits: A Survey of Abstention in Large Language Models. Survey of abstention methods for LLM systems, 2026.
- [16] AbstentionBench: Reasoning About When Language Models Should Abstain. Benchmark for abstention behavior in language models, 2025.
- [17] Fine-Grained Confidence Calibration of Large Language Models in Automated Code Revision. arXiv:2604.06723, 2026.
- [18] SGAgent: An Agentic Framework for Software Engineering Localization and Repair. arXiv:2602.23647, 2026.

Publication

No publication is being formally claimed from this thesis at the time of submission. If a paper, report, or accepted manuscript arises directly from this work before final submission, it may be listed here in the required institute format.

Author's Biography

Upamanyu Ghosh is a postgraduate student of **Computer Science and Engineering** in the **Department of Computer Science and Engineering, Dr. SPM International Institute of Information Technology, Naya Raipur**. His research interests include automated program repair, continuous-integration tooling, and the safe application of large language models to software-engineering tasks.